

宁波大學
NINGBO UNIVERSITY

《软件技术基础课程设计》

实验项目：2-3 综合编程实验

院　　系：计算机科学与技术系

学　　期：2024 年第 1 学期

班　　级：计算机科学与技术 2 班

学　　号：226001634、226004124、226001624

姓　　名：赖诗亮、崔智英、顾凯琪

指导教师：叶绪伦，李纲

日期：2024 年 12 月 24 日

目录

一、项目简介 3

1.1 项目背景 3

1.2 项目介绍 3

1.3 技术亮点 4

二、团队分工 4

三、开发环境 5

四、设计思路 5

4.1 需求分析 5

4.1.1 背景 5

4.1.2 功能性需求 6

4.1.3 非功能性需求 7

4.1.4 用例分析 9

4.1.5 顺序图 13

4.1.6 类图 14

4.2 总体设计 18

4.3 模块功能详解 19

4.4 各模块流程图 22

4.5 软件代码展示 26

4.5.1 ESP8266 芯片代码 26

4.5.2 LSTM 模型训练代码 29

4.5.3 LSTM 模型预测代码 34

4.5.4 后端主函数代码 39

4.5.5 相应后端 API 代码（部分） 40

4.5.6 前端主界面 44

4.5.7 前端 3D 模型展示页面 53

4.5.8 前端数据分析页面 79

4.5.9 温度折线图 96

4.5.10 湿度折线图 99

4.5.9 前端密码修改页面 102

五、实验运行结果展示 105

六、实验总结 109

一、 项目简介

1.1 项目背景

智慧物联 3D 农业大棚仿真平台。我们的项目以 Flask 和 Vue 前后端开发为基础，通过 DHT22 采集真实世界数据进行监测和预测，并结合了先进的 Three.js 技术构建仿真 3D 大棚，其内含的各项设备可通过温湿度等信息智能控制，为用户提供了智慧农业的新方案。

技术亮点：

软硬件结合：

- 数据采集：通过 DHT22 传感器获取环境的温湿度数据，并使用 Flask 后端处理数据采集、存储和分析，实现实时监控和历史数据回溯。
- 预测算法：利用机器学习模型 LSTM，对采集数据进行预测分析，为农业生产提供科学依据。

前端构建：

- Vue 框架前端构建：使用 Vue 框架设计前端界面，通过组件化开发实现模块化管理。JavaScript 作为主要开发语言，为用户提供直观流畅的界面和交互体验，例如实时数据显示、设备控制等功能。
- Three.js 仿真大棚：使用 Three.js 技术构建 3D 虚拟大棚，仿真展示温室内部布局、设备和环境变化。实现实时温湿度数据在 3D 场景中的动态可视化，通过颜色变化或模型运动反映温湿度变化。

后端 Flask 支持：

- 提供高效的 RESTful API 接口，与 Vue 前端进行数据通信。
- 负责 DHT22 数据的接收、处理和存储，支持实时监测和历史数据分析。
- 集成预测模型，响应前端的预测请求，返回未来一段时间内的环境数据。

1.2 项目介绍

本项目旨在打造一个智慧物联 3D 农业大棚仿真平台，通过传感设备实时采

集温室中的环境数据，进行监测和预测分析并对大棚中的设备进行仿真操控。平台提供直观的可视化界面，展现温室内部环境的变化情况，并支持用户根据数据调整管理策略。通过虚拟仿真技术，用户可以身临其境地观察温室环境的动态变化，提升管理效率。本项目为农业生产提供了科学化、现代化的解决方案，助力智能化农业的发展。

1.3 技术亮点

首先，项目通过结合 DHT22 传感器和 LSTM（长短期记忆网络）模型，实现了温湿度数据的实时采集和精准预测。DHT22 传感器能够准确地从现实世界获取温湿度数据，这些数据经过 LSTM 模型的处理与训练后，能够预测未来的温湿度变化趋势，帮助在农业大棚环境中实现精准的气候预测和调控。这一创新设计将传感器硬件与深度学习模型紧密结合，为智能农业提供了数据驱动的决策支持。

其次，项目利用 Three.js 构建了一个高度仿真的大棚模拟系统，通过虚拟环境中的温湿度数据，能够对大棚内的各类设备进行实时仿真控制。通过对温湿度等环境参数的变化，系统能够自动调整如通风、加热、灌溉等设备的状态，模拟并优化农业大棚的管理与运作。这一仿真系统不仅提升了对大棚环境管理的可视化和交互性，也为农业自动化控制提供了强大的支持，极大提高了温室大棚的生产效率和环境管理水平。

二、团队分工

赖诗亮：33.3%

负责系统后端的开发和维护以及 3D 部分，包括核心代码的编写。

实现游戏整体架构的后端逻辑，支持游戏流程正常运行。

设计并优化数据库与后端的交互逻辑，为前端提供稳定的数据接口服务。

调整和测试软硬件结合的数据处理逻辑，确保传感器数据的实时传输与存储。

崔智英：33.3%

负责系统前端的开发以及3D部分，使用 Vue 实现动态 UI 和用户交互界面。

为前端提供所需的图片素材，包括视频和对应图标。

设计并实现数据库数据在前端的展示，优化数据展示的实时性和可读性。

参与整体界面风格的设计，提升用户体验。

顾凯琪：33.3%

负责软硬件结合部分的开发和测试以及机器学习部分，确保传感设备的稳定运行。

设计存储记录数据的数据库表结构，实现用户数据的存储与上传功能。

优化传感器采集数据的处理逻辑，并与后端对接，确保数据准确传递。

配合完成仿真系统的软硬件参数设计和调整，提升系统整体性能。

三、开发环境

网页前端：Vue、Element UI、Three.js

后端开发工具：Flask

硬件设备：ESP8266、DHT22、SIM7020

数据库：Mysql

机器学习模型：LSTM

四、设计思路

4.1 需求分析

4.1.1 背景

环境温湿度监测在农业、物流、仓储、医疗等众多领域中具有重要意义。通过实时监测温湿度，不仅可以及时预警，采取有效措施以保障设备正常运行和产品质量，还能显著提高生产效率，防止潜在损失的发生。在全球气候变化和环境问题日益严峻的背景下，极端天气频发和气温波动加剧对许多行业带来了巨大的挑战。

当前，许多企业和机构仍采用传统的手工监测模式，依赖有限的监测点和人力资源，导致数据获取不及时、覆盖范围有限、准确性不足等问题。这种模式存在高成本、低效率、难以实现精细化管理等弊端，已无法满足现代化管理的需求。为解决这些问题，亟需一种智能化的环境温湿度监测报警与调控系统。该系统基

于物联网技术，能够通过部署传感器实时采集环境温湿度数据，并对数据进行处理和分析。系统不仅可以根据预设条件触发报警，还能结合设备联动调控功能，根据温湿度变化自动调整大棚内部设备（如通风、加湿、制冷等），从而保障环境稳定。此外，项目创新性地融入 3D 仿真技术，通过构建虚拟大棚，用户可直观、沉浸式观察环境动态变化及设备运行状态，为科学管理提供决策支持。这一解决方案将实现环境状态实时监测、异常情况自动响应、设备智能联动调控和仿真可视化展示的有机结合，为农业生产和其他相关行业提供智能化、现代化的管理手段，推动精细化和高效化管理的发展。

4.1.2 功能性需求

（一）实时监测

数据采集：通过分布在不同位置的无线传感节点（包含 ESP8266、SIM7020、DHT22 模

块）实时获取环境温湿度数据，每个节点独立工作且可分辨，互不干扰。

数据传输与汇总：节点将采集到的数据（添加节点编号、时间戳等信息）打包，通过 NB-IoT 网络发送至 EMQX 服务器的特定主题，实现信息的汇总处理，供后续软件端获取。

（二）数据分析与渲染呈现

数据处理：后端程序接收传感器数据后，进行数据的存储、分析等操作，如筛选异常数据、计算平均值等统计分析。

动态报表展示：前端以动态报表形式呈现温湿度信息，包括二维折线图、三维热力图等直观展示方式，展示阈值范围与平均温度线，报表每隔 1 秒向后端请求新增数据以实现动态更新，且能确保多个设备数据展示的独立性。

（三）异常报警设置与通知

报警规则自定义：系统支持管理员在设备管理界面设置温湿度阈值上下限，实现报警规则的自定义。

报警触发与通知：当温湿度数据超出设定阈值，系统自动触发报警机制，并以短信、电子邮件等形式通知相关人员。

（四）历史数据查询

数据存储与记录：系统将一定时间范围内的历史温湿度数据存储在 MySQL 数据库中，

数据库设计了记录表用于存储详细数据，包括记录 ID、来源设备、时间戳信息、温度信息、湿度信息等字段。

查询功能实现：用户可通过前端界面输入查询条件（如时间范围、设备 ID 等），前

端向后端发送请求，后端从数据库中检索符合条件的数据并返回给前端进行展示。

（五）日志报告导出

报告生成：系统按指定时间间隔生成报告，内容包括数据分析表、异常情况统计等。

导出功能：在历史数据界面，用户可点击按钮将所有数据或异常数据导出为 Excel 表格，方便后续数据处理与分析。

（六）设备管理

设备信息显示：在设备管理界面展示所有导入系统的无线节点信息，如设备 ID、设备

型号、设备状态、发送数据流、温度阈值上下限、湿度阈值上下限、设备描述等。

设备操作功能：支持管理员对设备信息进行编辑、删除操作，同时可添加新设备，添加时系统自动监测设备 ID 的唯一性，确保数据准确整合与前端渲染。

4.1.3 非功能性需求

非功能性需求通常涉及系统性能、安全性、交互流畅性等方面。本系统的非功能性需求大致包括以下几个方面：

（1）系统性能

系统的性能指标主要包含响应时间、吞吐量等方面。系统应具备较低的响应时间，及时处理和展示温湿度数据，确保用户能够快速获取实时信息；同时系统应具备较大的吞吐量，便于实现多点监测，将收集到的数据批量处理分析存储。

（2）可拓展性

系统应支持水平扩展，并能够根据需求动态添加或移除传感器设备，以满足不同规模和复杂度的环境监测需求。

（3）安全性

系统应保证传输的温湿度数据在传输过程中的隐私和保密性，采用合适的加密技术确保传输的安全性；同时系统必须保证系统的稳定性，避免出现系统崩溃或数据丢失等问题。

（4）可靠性

系统应具备故障恢复和容错机制，以确保在部分设备或系统故障的情况下，仍能继续正常运行和监测环境温湿度，此外应保持稳定和可靠的运行，减少系统崩溃或中断的概率，确保数据的准确性和一致性。

（5）交互流畅性

系统应具备直观、易用的用户界面，能够清晰地呈现各项核心的功能模块以及实时数据，能让用户能够快速上手操作和浏览温湿度数据，同时面向用户提供清晰的文档和培训资料，以便用户能够准确理解和使用系统功能。

4.1.4 用例分析

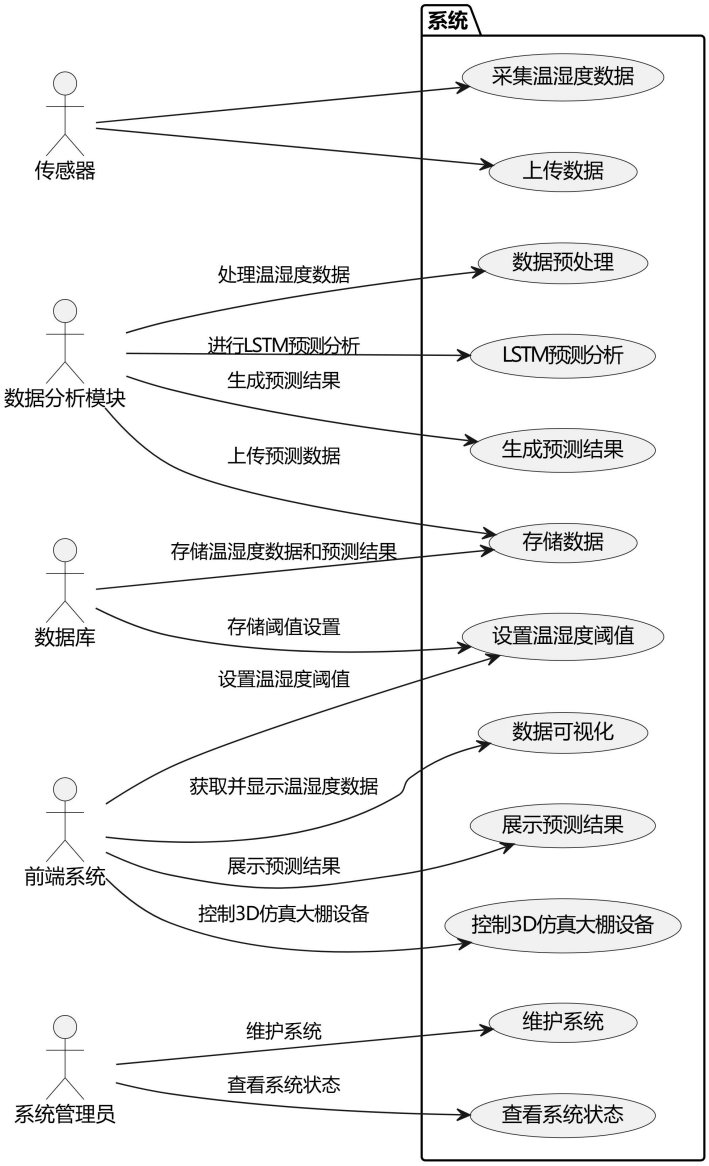


图 1 用例分析图

用例名称：实时数据采集与上传

参与者：传感器、数据分析系统、数据库

目标：

系统实时采集温湿度数据，并将数据上传到数据库。

触发条件：

- 系统启动后，传感器开始工作，触发数据采集。

主流事件：

1. **传感器采集数据：**

- 传感器实时采集温湿度数据。

2. **数据传输到分析系统：**
   - 采集到的数据传输到数据分析系统，进行初步分析。
3. **数据上传至数据库：**
   - 数据分析系统将分析后的数据上传到数据库进行存储。

其他场景：

1. **传感器失效：**
   - 如果传感器未能采集到数据，系统会提示传感器故障，并尝试重新连接。
2. **数据上传失败：**
   - 数据上传至数据库时发生网络故障，系统会尝试重新连接，并给出失败提示。

用例名称：数据分析与 LSTM 预测

参与者：数据分析系统、LSTM 模型、预测系统

目标：

使用 LSTM 模型分析温湿度数据，并进行未来趋势预测。

触发条件：

- 当新数据上传至数据库时，数据分析系统触发 LSTM 模型进行预测。

主流事件：

1. **数据接入 LSTM 模型：**
   - 数据分析系统接收温湿度数据，并将其输入 LSTM 模型进行训练和预测。
2. **LSTM 模型进行数据预测：**
   - LSTM 模型根据历史数据进行温湿度未来值的预测。
3. **预测结果保存并返回：**
   - 预测结果保存，并将预测数据返回给前端系统以进行可视化。

其他场景：

1. **LSTM 模型训练失败：**

- 若 LSTM 模型在训练过程中失败（如数据不符合要求），系统将提示并记录错误。

2. **预测数据异常：**
   - 预测结果异常（如温湿度预测值过高或过低），系统会重新训练模型，并通过反馈机制进行调整。

用例名称：前端数据可视化

参与者：前端用户、数据展示系统、数据库

目标：

用户通过前端界面查看实时温湿度数据和预测结果。

触发条件：

- 用户访问前端页面，系统加载数据并展示。

主流事件：

1. **用户请求查看数据：**
   - 用户通过前端界面请求查看温湿度数据。
2. **系统从数据库获取数据：**
   - 数据展示系统从数据库中获取实时数据和预测数据。
3. **前端展示数据：**
   - 系统将数据通过可视化组件展示在前端界面，包括图表、趋势线等。

其他场景：

1. **数据加载失败：**
   - 若数据库查询失败，前端展示系统将提示“数据加载失败，请稍后再试”。
2. **前端无数据显示：**
   - 若没有可显示的数据（如数据库为空或出现故障），系统会给出提示“当前没有数据”。

用例名称：设备控制与 3D 仿真

参与者：前端用户、设备控制系统、仿真系统

目标：

根据实时温湿度数据，前端控制 3D 仿真大棚设备的工作状态。

触发条件：

- 当温湿度数据达到一定阈值时，触发设备控制。

主流事件：

1. **温湿度数据分析：**
    - 前端系统分析当前温湿度数据，判断是否超出预设阈值。
2. **设备控制指令发送：**
    - 如果温湿度数据不符合要求，系统会发送控制指令，调节大棚内设备（如风扇、加湿器、灯光等）。
3. **仿真展示设备状态：**
    - 仿真系统根据设备状态调整仿真大棚的内部设备工作状况，并在前端显示。

其他场景：

1. **设备控制失败：**
    - 如果设备未能响应控制指令，系统将提示“设备控制失败，请检查设备状态”。
2. **仿真显示异常：**
    - 若仿真系统显示异常，系统会提示“仿真显示错误，请检查仿真系统”。

4.1.5 顺序图

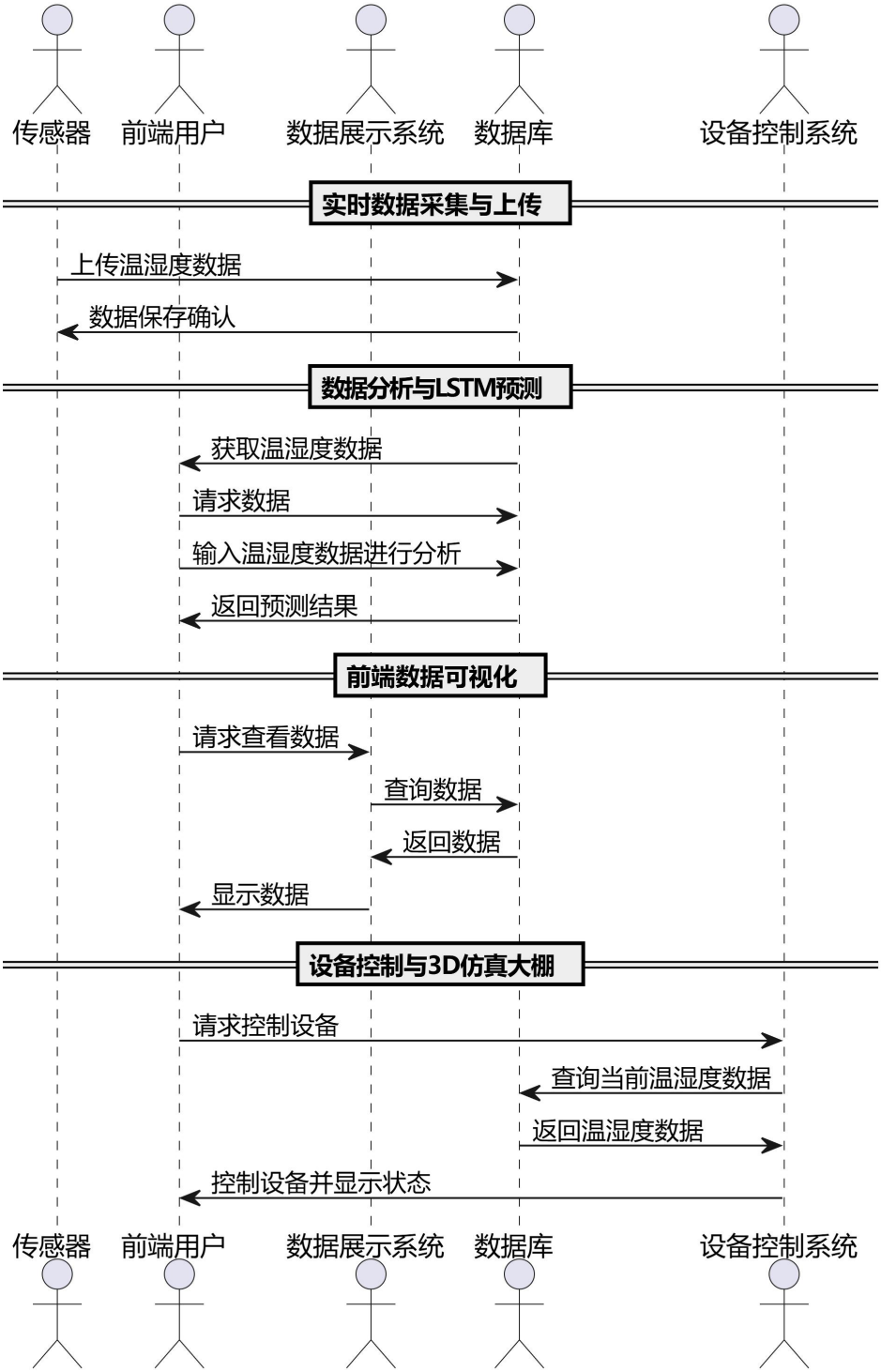


图 2 顺序图

4.1.6 类图

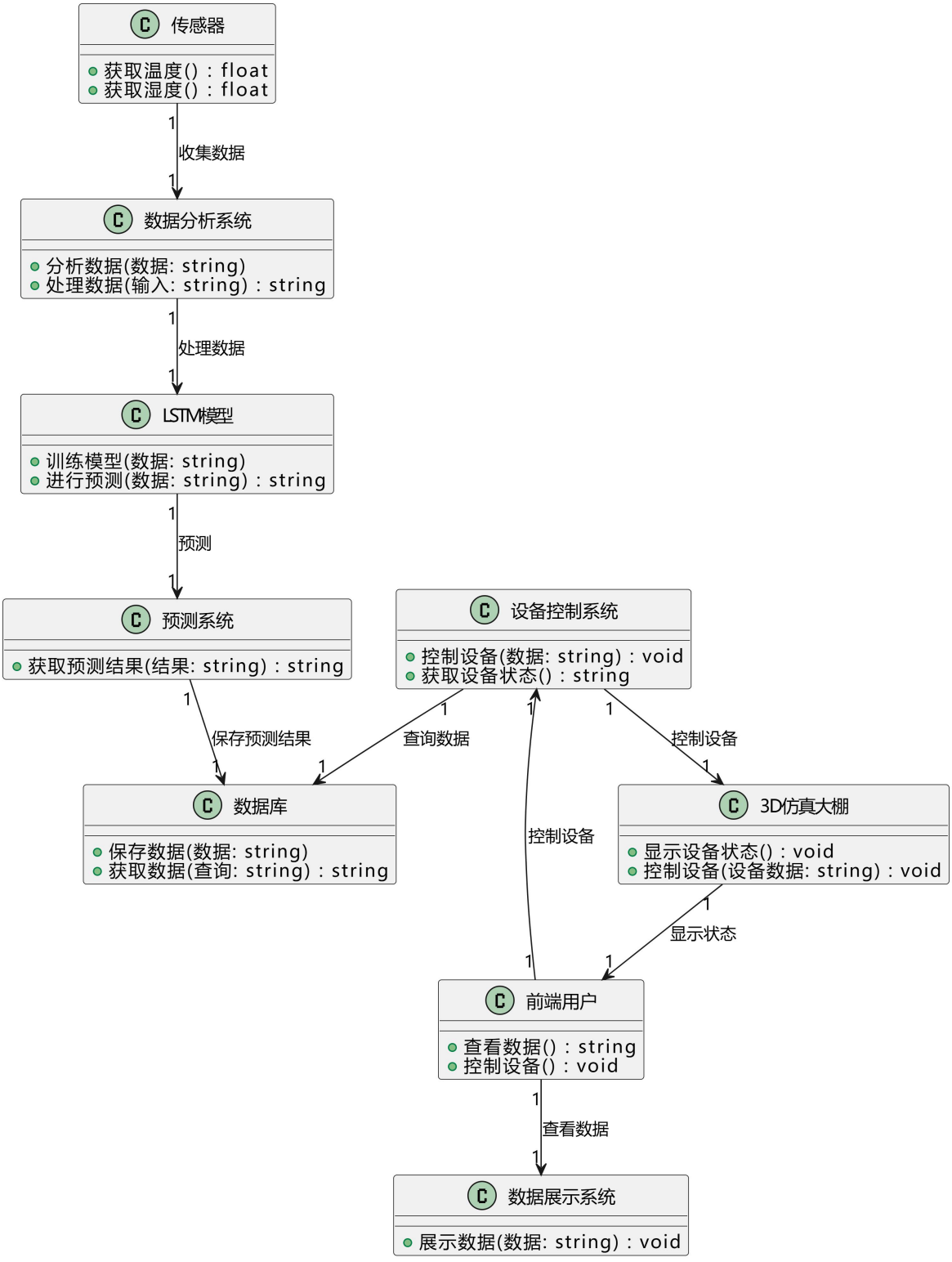

传感器
获取温度() : float
获取湿度() : float
1
收集数据
1
数据分析系统
分析数据(数据: string)
处理数据(输入: string) : string
1
处理数据
1
LSTM模型
训练模型(数据: string)
进行预测(数据: string) : string
1
预测
1
预测系统
获取预测结果(结果: string) : string
1
保存预测结果
1
设备控制系统
控制设备(数据: string) : void
获取设备状态() : string
1
查询数据
1
1
控制设备
1
数据库
保存数据(数据: string)
获取数据(查询: string) : string
控制设备
3D仿真大棚
显示设备状态() : void
控制设备(设备数据: string) : void
1
显示状态
1
1
前端用户
查看数据() : string
控制设备() : void
1
查看数据
1
数据展示系统
展示数据(数据: string) : void

图 3 类图

1. **传感器（Sensor）**
   - **职责：**
     - 传感器负责实时采集温湿度数据，为整个系统提供环境数据。
   - **方法：**
     - 获取温度()：获取当前环境温度。
     - 获取湿度()：获取当前环境湿度。
   - **与其他类的关系：**
     - 传感器会将采集到的温湿度数据传输给 **数据库** 类进行存储。
2. **数据库（Database）**
   - **职责：**
     - 数据库负责存储传感器采集的数据、分析结果以及设备控制信息。
   - **方法：**
     - 保存数据(数据: string)：将传感器采集的数据或分析结果保存到数据库中。
     - 获取数据(查询: string)：根据传入的查询条件从数据库中获取数据。
   - **与其他类的关系：**
     - **传感器** 将数据保存到 **数据库**。
     - **数据分析系统** 和 **预测系统** 从 **数据库** 获取数据进行分析与预测。
     - **设备控制系统** 查询数据库获取温湿度等数据，进行设备控制。
     - **3D 仿真大棚** 和 **前端用户** 也可以从 **数据库** 获取设备状态或其他必要数据。
3. **数据分析系统（AnalysisSystem）**
   - **职责：**
     - 数据分析系统负责对传感器采集到的数据进行预处理或基础分析，为后续的预测做好准备。
   - **方法：**
     - 分析数据(数据: string)：对传感器数据进行基础的分析处理。

- 处理数据(输入: string)：进一步处理和整理数据，为 LSTM 模型提供输入。

- **与其他类的关系：**
  - **传感器** 将采集到的数据传递给 **数据分析系统** 进行初步分析。
  - 分析后的数据会传递给 **LSTM 模型** 进行更深入的预测分析。

4. **LSTM 模型（LSTMModel）**
   - **职责：**
     - 该模块负责通过 LSTM（长短期记忆）模型对历史温湿度数据进行训练和预测，生成未来的温湿度预测结果。
   - **方法：**
     - 训练模型(数据: string)：用传感器数据训练 LSTM 模型。
     - 进行预测(数据: string)：使用训练好的 LSTM 模型进行预测，并生成结果。
   - **与其他类的关系：**
     - **数据分析系统** 会将处理后的数据传递给 **LSTM 模型** 进行训练。
     - **LSTM 模型** 会根据训练结果提供预测输出，供 **预测系统** 使用。
5. **预测系统（PredictionSystem）**
   - **职责：**
     - 该模块负责处理 LSTM 模型生成的预测结果,进一步进行格式化或逻辑处理，为 **设备控制系统** 和 **前端用户** 提供预测数据。
   - **方法：**
     - 获取预测结果(结果: string)：获取并格式化 LSTM 模型的预测结果。
   - **与其他类的关系：**
     - **LSTM 模型** 会将预测结果传递给 **预测系统**，该模块会进行必要的处理。
     - 预测结果可以保存到 **数据库** 中，并提供给 **设备控制系统** 或 **前端用户**。
6. **前端用户（FrontendUser）**

- **职责：**
  - 前端用户模块代表系统的最终用户，用户可以查看分析和预测结果，控制大棚设备。
- **方法：**
  - 查看数据()：用户可以通过该方法查看温湿度数据及其预测结果。
  - 控制设备()：用户通过该方法控制 3D 仿真大棚中的设备。
- **与其他类的关系：**
  - **前端用户** 可以从 **数据展示系统** 查看数据。
  - **前端用户** 可以通过 **设备控制系统** 控制 3D 仿真大棚中的设备，并查看设备状态。

7. **设备控制系统（DeviceControlSystem）**
   - **职责：**
     - 设备控制系统负责根据预测系统提供的预测结果或实时温湿度数据，控制温室内的各类设备（如灯光、温控器、湿度调节设备等）。
   - **方法：**
     - 控制设备(数据: string)：根据传入的数据控制设备（如调整温度、湿度等）。
     - 获取设备状态()：获取设备当前的状态。
   - **与其他类的关系：**
     - **设备控制系统** 通过 **数据库** 获取温湿度数据或设备状态，进行设备控制。
     - **前端用户** 可以通过 **设备控制系统** 控制设备。
     - **设备控制系统** 会向 **3D 仿真大棚** 发送控制信号，控制大棚内的设备。
8. **3D 仿真大棚（3DGreenhouse）**
   - **职责：**
     - 3D 仿真大棚用于将温室内设备的状态以可视化的形式展示给前端用户。用户可以通过此系统查看设备的状态和控制设备的工作情况。
   - **方法：**

- 显示设备状态()：显示当前设备的状态。
- 控制设备(设备数据: string)：根据设备控制系统的指令控制设备。

- **与其他类的关系：**
  - **3D 仿真大棚** 可以接收来自 **设备控制系统** 的指令，控制设备。
  - **前端用户** 通过 **3D 仿真大棚** 查看设备状态。

4.2 总体设计

该项目采用前后端分离，整个系统可以划分为三个模块，分别为：硬件模块，前端模块，后端模块。其中硬件模块可以细分为温度和湿度检测模块，前端模块细分为 3D 大棚模块、温湿度可视化模块和报表导出模块，后端模块可分为数据库连接模块和数据训练预测模块。

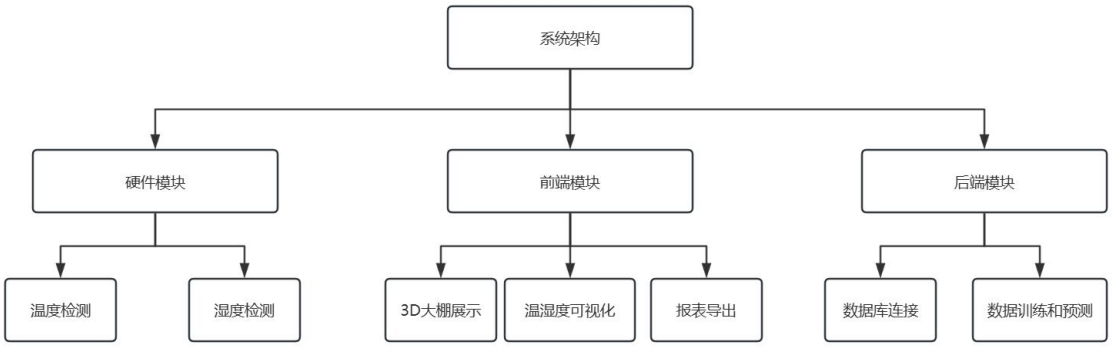


图 4 系统总模块设计图

数据库 E-R 图设计：

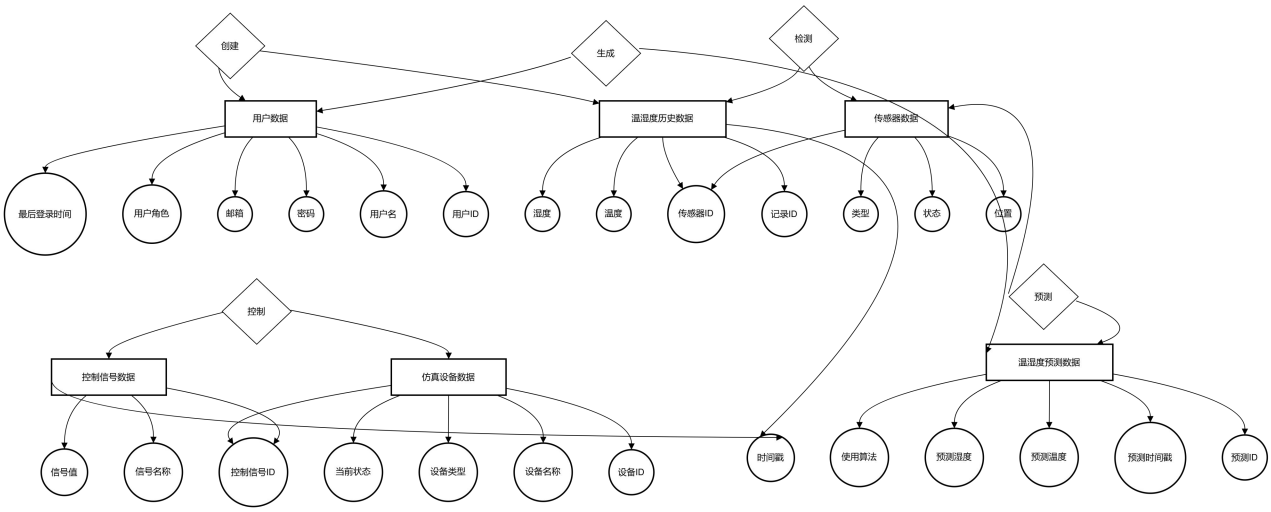


图 5 E-R 图设计

4.3 模块功能详解

表 1 用户模块设计

功能模块	包含子功能模块	功能
硬件模块	温度检测模块	本模块结合 DHT22 传感器和 ESP8266 开发板，实时采集环境温湿度数据，并通过 Wi-Fi 将数据传输至云端或本地服务器。传感器精确测量温湿度，ESP8266 支持 HTTP、MQTT 等协议，确保数据兼容性。模块支持实时监控、历史数据存储、查询功能，并能在温湿度超标时触发报警通知。具备低功耗模式，延长电池寿命，并支持多个传感器接入，适用于智能家居和环境监控等应用。
	湿度检测模块	本模块结合 DHT22 传感器和 ESP8266 开发板，实时采集环境温湿度数据，并通过 Wi-Fi 将数据传输至云端或本地服务器。传感器精确测量温湿度，ESP8266 支持 HTTP、MQTT 等协议，确保数据兼容性。模块支持实时监控、历史数据存储、查询功能，并能在温湿度超标时触发报警通知。具备低功耗模式，延长电池寿命，并支持多个传感器接入，适用于智能家居和环境监控等应用。

表 2 前端模块设计

功能模块	包含子功能模块	功能
前端模块	3D 大棚模块	本模块使用 Three.js 构建 3D 大棚模拟仿真，动态模拟大棚内的物理环境、植物生长和光照

		变化等。首先，通过 Three.js 创建大棚的 3D 模型，包括大棚框架、屋顶、地面等基础结构，并应用纹理以提高视觉真实感。接着，模拟大棚内的物理环境，如温度、湿度和气流的变化，通过动态调整这些环境参数影响植物的生长。使用动画技术来模拟植物在不同环境下的生长过程，包括叶片和茎干的变化。光照方面，通过调整太阳光强度和角度，模拟一天 24 小时内的光照变化，影响植物的光合作用和生长状态。整个系统实现了大棚内环境与植物生长之间的实时交互与仿真。
	温湿度可视化模块	本模块使用 Element UI 对传感器获取的温湿度数据进行可视化监视和预测。通过图表、仪表盘等组件，实时展示温湿度变化趋势，帮助用户直观了解环境数据。模块支持数据的动态更新，用户可通过界面查看历史数据并进行趋势分析，预测未来的温湿度变化。Element UI 提供的灵活布局和交互功能，使得数据展示更加直观和易于操作。同时，结合图表组件，用户可设置报警阈值，实时监测并及时得到温湿度异常的提醒。
	报表导出模块	本模块实现了从数据库中导出历史数据和异常数据，并将其以 Excel 格式保存。首先，通过数据库查询，提取符合条件的历史数据和异常数据（例如温湿度超标的记录）。然后，使用库如 pandas 或 openpyxl 来处理数据，将查询结果整理成表格格式。最终，生成 Excel 文件并提供下载功能，确保数据的可视化和存档。模块支持按时间、数据类型等多维度筛选

		数据,用户可以选择具体的时间范围或异常阈值来导出相关数据,方便进行后续分析或报告生成。

表 3 后端模块设计

功能模块	包含子功能模块	功能
后端模块	数据库连接	本模块使用数据库高效交互,动态管理用户信息、控制信息以及温湿度历史与预测数据的传递。首先,通过配置文件加载数据库连接信息,包括主机、端口、用户名、密码和数据库名称等,并通过连接池技术实现资源的动态分配与管理,确保高并发情况下的稳定性。通过模块提供的查询接口实现用户信息的管理(如登录验证和权限查询)、控制指令的存储与读取(包括用户发送的调控指令与设备反馈状态),以及温湿度数据的动态存储与预测结果的查询。通过时间区间筛选,支持温湿度历史数据的查询和趋势分析。整个系统实现了对用户、控制信息以及温湿度数据的实时管理与交互。
	数据训练预测模块	本模块使用机器学习算法（LSTM）对大棚温湿度历史数据进行训练,并生成未来的温湿度预测。首先,从数据库中提取历史温湿度数据并进行清洗处理。然后,选择合适的算法进行模型训练,并通过划分训练集和验证集来优化模型性能。训练完成后,模型根据实时数据生成温湿度预测,提供未来时间段的温湿度值。最后,模型通过评估和优化,保证预测结果的

		准确性，并定期更新以适应新的数据

4.4 各模块流程图

用户登录与注册流程图：

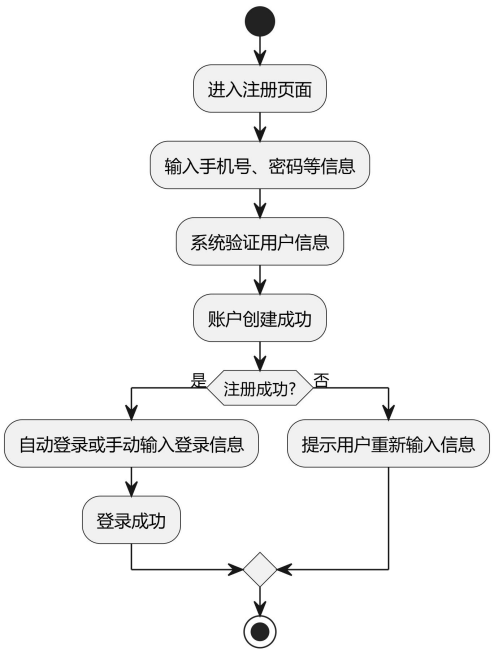


图 6 用户登录与注册流程图

传感器采集数据流程图：

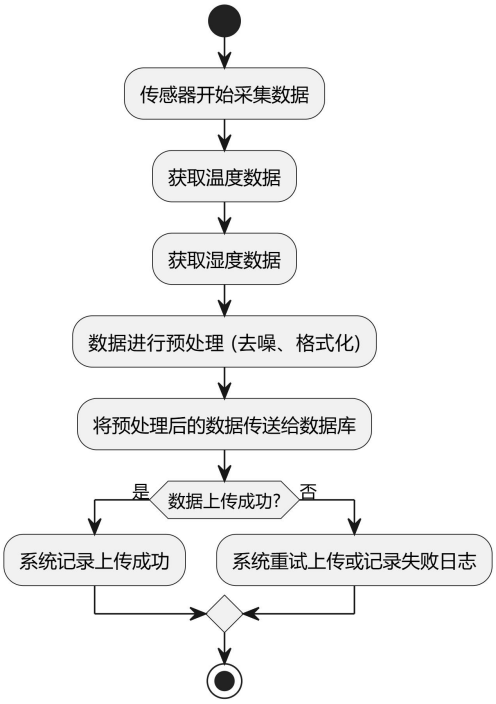


图 7 传感器采集数据图

数据分析与处理流程图：

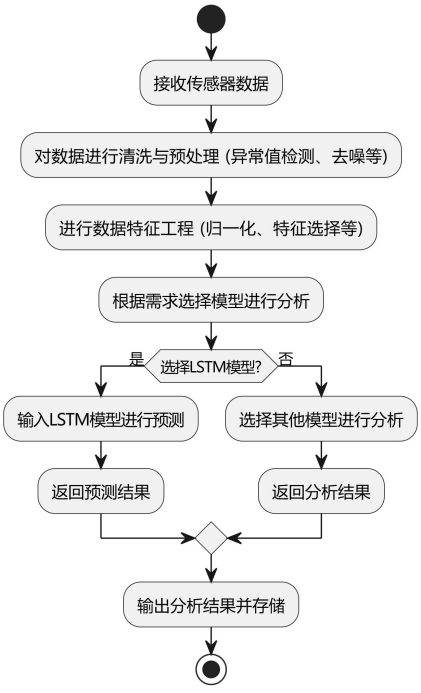


图 8 数据分析与处理图

LSTM 模型训练与预测流程图：

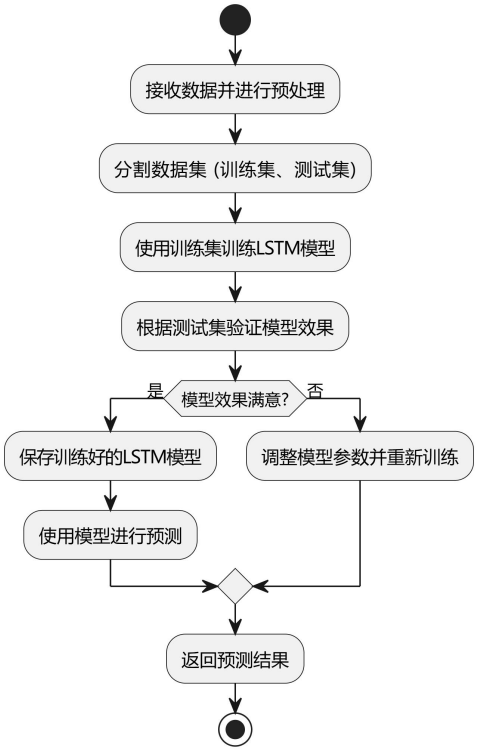


图 9 LSTM 模型训练与预测图

设备控制流程图：

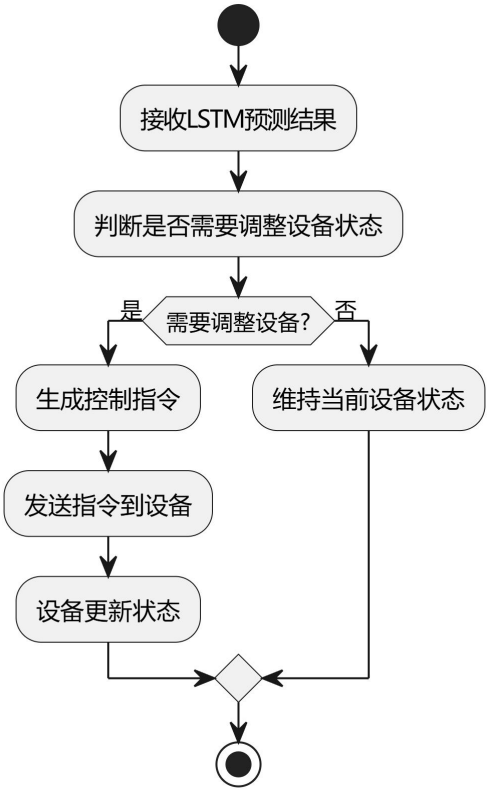


图 10 设备控制图

3D 仿真大棚流程图

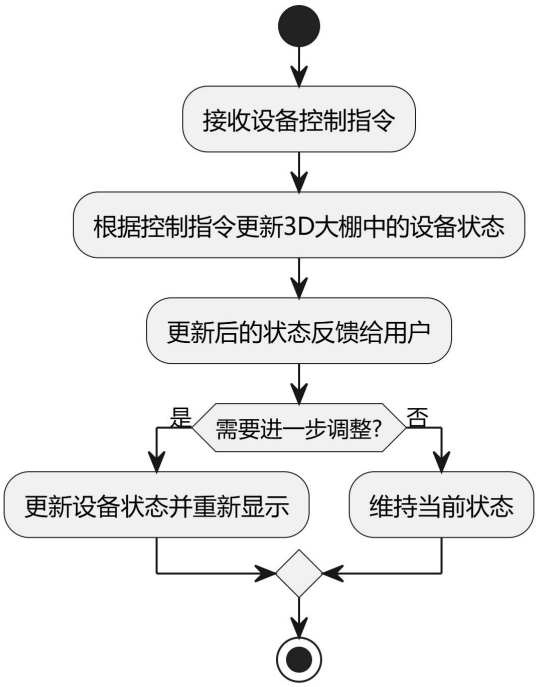


图 11 3D 仿真大棚图

4.5 软件代码展示

4.5.1 ESP8266 芯片代码

```
#include <DHT.h>
#include <ESP8266WiFi.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
float currentTemperature = 0;
float currentHumidity = 0;
unsigned long currentUnixTime = 0;

// 获取温湿度信息
void getTH() {
    delay(60000);    // 采样间隔：60 秒
    float RH = dht.readHumidity();
    float T = dht.readTemperature();
    currentTemperature = T;
    currentHumidity = RH;
}

// 封装成 JSON 格式
char messageBuff[1024];
void getJson() {
    StaticJsonDocument<1024> doc;
    JsonObject root = doc.to<JsonObject>();
    root["Temperature"] = currentTemperature;
    root["Humidity"] = currentHumidity;
    root["ID"] = 2;
    root["Timestamp"] = currentUnixTime;
    serializeJson(doc, messageBuff);
}

// WiFi 连接信息
const char* ssid = "kkaiqi";
const char* password = "15975300";

```

```
// WiFi 连接
void connectWiFi() {
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected.");
}

// MQTT Broker
const char* mqttServer = "10.83.11.4";
const int mqttPort = 1883;
const char* topic = "esp8266";
WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

// MQTT 服务器连接
void connectMQTTServer() {
    String clientID = "ESP8266-" + WiFi.macAddress();
    if (mqttClient.connect(clientID.c_str())) {
        Serial.println("MQTT Server Connected.");
    } else {
        Serial.print("MQTT Server Connect Failed. Client State: ");
        Serial.println(mqttClient.state());
        delay(3000);
    }
}

// 信息发布
void publishMQTTmsg() {
    getJson();
    if (mqttClient.publish(topic, messageBuff)) {
        Serial.println("Message Sent.");
    } else {
        Serial.println("Message Sending Failed.");
    }
    memset(messageBuff, 0, 1024);
}

// NTP 参数
const char* ntpServerName = "pool.ntp.org";
const int timeZone = 0;
```

```
WiFiUDP udp;
NTPClient timeClient(udp, ntpServerName, timeZone * 60);

// 获取 Unix 时间
void getUnixTime() {
    timeClient.update();
    unsigned long unixTime = timeClient.getEpochTime();
    currentUnixTime = unixTime > currentUnixTime ? unixTime : currentUnixTime + 3;
}

// 打印测试数据
void dataTest() {
    Serial.print("Unix Time: ");
    Serial.println(currentUnixTime);
    Serial.print("Temperature: ");
    Serial.println(currentTemperature);
    Serial.print("Humidity: ");
    Serial.println(currentHumidity);
}

void setup() {
    Serial.begin(9600);
    dht.begin();
    connectWiFi();
    timeClient.begin();
    mqttClient.setServer(mqttServer, mqttPort);
    mqttClient.setKeepAlive(120); // 设置 MQTT 心跳时间
    connectMQTTServer();
}

void loop() {
    if (mqttClient.connected()) {
        mqttClient.loop(); // 保持 MQTT 连接
        static unsigned long lastSampleTime = 0;
        if (millis() - lastSampleTime > 60000) { // 每 60 秒采样一次
            lastSampleTime = millis();
            getTH();
            getUnixTime();
            dataTest();
            publishMQTTmsg();
        }
    } else {
```

```
125.             connectMQTTServer();
126.         }
127.     }
128.
```

初始化库：

- 代码引入了多个库，用于温湿度传感器读取（DHT.h）、Wi-Fi 连接（ESP8266WiFi.h）、时间同步（NTPClient.h）、MQTT 通信（PubSubClient.h）、JSON 数据处理（ArduinoJson.h）。

DHT 传感器数据采集：

- 使用 dht.readHumidity() 和 dht.readTemperature() 来获取温湿度数据，并保存到 currentHumidity 和 currentTemperature 变量中。

Wi-Fi 连接：

- connectWiFi() 函数负责连接到 Wi-Fi 网络，直到连接成功为止。

NTP 时间获取：

- 通过 NTP 协议获取当前的 Unix 时间戳，使用 timeClient.getEpochTime() 获取时间戳，并保存在 currentUnixTime 中。

MQTT 数据发布：

- 通过 mqttClient.publish() 将温湿度数据和时间戳以 JSON 格式发送到 MQTT 服务器。

数据封装：

- getJson() 函数将温湿度数据、设备 ID 和时间戳封装成 JSON 格式，便于通过 MQTT 传输。

循环操作：

- loop() 函数在每次循环中检查 MQTT 连接状态，如果连接已建立，则每 60 秒采集一次温湿度数据，并将数据发布到 MQTT 服务器；如果连接失败，则重新连接到 MQTT 服务器。

4.5.2 LSTM 模型训练代码

```
1.  import  torch
```

```
import torch.nn as nn
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler    # 使用标准化
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt


# LSTM 模型定义（使用双向 LSTM 和 Dropout）
class LSTMModel(nn.Module):
        def __init__(self, input_size, hidden_layer_size, output_size, num_layers=2, dropout=0.3, bidirectional=True):
                super(LSTMModel, self).__init__()
                # 添加双向 LSTM
                self.lstm = nn.LSTM(input_size, hidden_layer_size, num_layers=num_layers,
                                    batch_first=True, dropout=dropout, bidirectional=bidirectional)
                # 计算输出尺寸，双向 LSTM 会增加 2 倍的隐藏层单元数
                self.fc = nn.Linear(hidden_layer_size * 2 if bidirectional else hidden_layer_size, output_size)

        def forward(self, x):
                lstm_out, (hn, cn) = self.lstm(x)
                predictions = self.fc(lstm_out[:, -1])    # 仅使用最后时刻的输出进行预测
                return predictions


# 从 Excel 读取数据
def read_data_from_excel(file_path):
        df = pd.read_excel(file_path)
        df = df.dropna(subset=["Temperature", "Humidity", "Timestamp"])    # 去掉缺失值
        return df[["Temperature", "Humidity", "Timestamp"]]


# 数据预处理（只使用温度和湿度）
def prepare_data(data, time_step):
        scaler = StandardScaler()    # 使用标准化

        # 只保留需要的特征（温度和湿度）
        features = ["Temperature", "Humidity"]
        scaled_data = scaler.fit_transform(data[features])
```

```
40.
41.         #  创建时间序列数据集
42.         X,  y  =  [],  []
43.         for  i  in  range(len(scaled_data)  -  time_step):
44.                 X.append(scaled_data[i:i  +  time_step])
45.                 y.append(scaled_data[i  +  time_step,  :])    #  预测的是温度和湿度
46.
47.         X  =  np.array(X)
48.         y  =  np.array(y)
49.         print(X.shape)
50.
51.         return  X,  y,  scaler
52.
53.
54. #  训练函数
55. def  train_model(model,  train_X,  train_y,  epochs=200,  batch_size=64,  learning_rate=0.0005):    #  调整学习率
56.         train_X  =  torch.tensor(train_X,  dtype=torch.float32)
57.         train_y  =  torch.tensor(train_y,  dtype=torch.float32)
58.
59.         criterion  =  nn.MSELoss()
60.         optimizer  =  torch.optim.Adam(model.parameters(),  lr=learning_rate)
61.
62.         #  训练过程
63.         for  epoch  in  range(epochs):
64.                 model.train()
65.                 optimizer.zero_grad()
66.
67.                 #  Forward  pass
68.                 output  =  model(train_X)
69.
70.                 #  计算损失
71.                 loss  =  criterion(output,  train_y)
72.
73.                 #  反向传播
74.                 loss.backward()
75.                 optimizer.step()
76.
77.                 #  每 10 个 epoch 打印一次损失
78.                 if  (epoch  +  1)  %  10  ==  0:
79.                         print(f'Epoch  [{epoch  +  1}/{epochs}],  Loss:  {loss.item():.4f}')
```

```
80.
81.         #  保存训练后的模型
82.         torch.save(model.state_dict(),  "model.pth")
83.         print("Model  training  complete.  Model  saved  as  model.pth."
    )
84.
85.
86. #  读取  Excel  数据并进行训练
87. file_path  =  r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
    \Intelligent-temperature-and-humidity-system\public\mqtt_data.xlsx"
88. data  =  read_data_from_excel(file_path)
89.
90. #  数据预处理
91. time_step  =  10
92. X,  y,  scaler  =  prepare_data(data,  time_step)
93.
94. #  构建训练集和验证集
95. train_X,  val_X,  train_y,  val_y  =  train_test_split(X,  y,  test_size
    =0.2,  shuffle=False)
96.
97. #  转换为  PyTorch  tensor
98. train_X  =  torch.tensor(train_X,  dtype=torch.float32)
99. train_y  =  torch.tensor(train_y,  dtype=torch.float32)
100.  val_X  =  torch.tensor(val_X,  dtype=torch.float32)
101.  val_y  =  torch.tensor(val_y,  dtype=torch.float32)
102.
103.  #  初始化  LSTM  模型，调整  input_size
104.  model  =  LSTMModel(input_size=2,  hidden_layer_size=200,  output_size
    =2,  num_layers=3,  dropout=0.1)
105.
106.  #  训练模型
107.  train_model(model,  train_X,  train_y,  epochs=800,  learning_rate=0.0
    01)
108.
109.  #  预测和评估
110.  model.eval()
111.  with  torch.no_grad():
112.          predictions  =  model(val_X)
113.
114.  #  逆归一化并显示结果
115.  predictions  =  scaler.inverse_transform(np.hstack([predictions.numpy()
    ,  np.zeros((predictions.shape[0],  0))]))
116.  val_y  =  scaler.inverse_transform(np.hstack([val_y.numpy(),  np.zeros
    ((val_y.shape[0],  0))]))
```

```
117.
118.  #  打印评估结果
119.  print(f"预测值:  {predictions[:5]}")
120.  print(f"实际值:  {val_y[:5]}")
121.
122.  #  可视化预测结果
123.  plt.plot(val_y[:,  0],  label='True  Temperature')
124.  plt.plot(predictions[:,  0],  label='Predicted  Temperature')
125.  plt.legend()
126.  plt.show()
127.
```

LSTM 模型：

- 使用了双向 LSTM（bidirectional=True）和 Dropout 层（dropout=0.3）来避免过拟合。
- LSTM 层输出后，使用一个全连接层（nn.Linear）进行最终的预测。

数据预处理：

- 使用 StandardScaler 对温度和湿度数据进行标准化。
- prepare_data 函数会根据给定的时间步（time_step）创建时间序列数据集。

训练函数：

- 在 train_model 函数中使用 Adam 优化器和 MSE 损失函数来训练模型。
- 训练过程中每 10 个 epoch 输出一次损失。

读取数据：

- read_data_from_excel 函数从指定路径的 Excel 文件中读取温湿度数据，并去掉缺失值。

模型训练：

- 使用训练数据进行训练，并保存训练后的模型（model.pth）。

预测和评估：

- 在训练后，使用验证集（val_X）进行预测，并与实际值进行比较。
- 通过反归一化处理，输出真实和预测的温度数据。

可视化：

- 使用 matplotlib 绘制实际温度与预测温度的对比图。

4.5.3 LSTM 模型预测代码

```
import paho.mqtt.client as mqtt
import json
import time
from openpyxl import Workbook, load_workbook
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.preprocessing import MinMaxScaler
from datetime import datetime, timedelta

# 服务器和主题信息
host = '10.20.196.78'
port = 1883
topic = 'esp8266'

# 创建 Excel 工作簿和工作表（用于存储原始数据）
def create_mqtt_data_excel():
        wb = Workbook()
        ws = wb.active
        ws.title = "Sensor Data"
        ws.append(["Temperature", "Humidity", "deviceId", "Timestamp"]
)    # 设置表头
        wb.save(
                r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
\Intelligent-temperature-and-humidity-system\public\mqtt_data.xlsx")
# 保存为 Excel 文件

# 创建 Excel 工作簿和工作表（用于存储预测数据）
def create_predicted_data_excel():
        wb = Workbook()
        ws = wb.active
        ws.title = "Predicted Data"
        ws.append(["Temperature", "Humidity", "deviceId", "Timestamp"]
)    # 设置表头
        wb.save(
                r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
\Intelligent-temperature-and-humidity-system\public\predicted_data.xlsx")
    # 保存为 Excel 文件

# LSTM 模型定义（与训练部分兼容）
```

```
class LSTMModel(nn.Module):
        def __init__(self, input_size, hidden_layer_size, output_size, num_layers=2, dropout=0.3, bidirectional=True):
                super(LSTMModel, self).__init__()
                # 添加双向 LSTM
                self.lstm = nn.LSTM(input_size, hidden_layer_size, num_layers=num_layers,
                                                                batch_first=True, dropout=dropout, bidirectional=bidirectional)
                self.fc = nn.Linear(hidden_layer_size * 2 if bidirectional else hidden_layer_size, output_size)

        def forward(self, x):
                lstm_out, (hn, cn) = self.lstm(x)
                predictions = self.fc(lstm_out[:, -1])    # 使用最后时间步的输出进行预测
                return predictions

# 构建 LSTM 模型
def build_model():
        model = LSTMModel(input_size=2, hidden_layer_size=200, output_size=2, num_layers=3, dropout=0.3)
        model.load_state_dict(torch.load("model.pth"))    # 加载预训练模型
        model.eval()    # 设置为评估模式
        return model

# 初始化 LSTM 模型
model = build_model()

# 数据归一化
scaler = MinMaxScaler(feature_range=(0, 1))

# 预测函数：预测未来 60 个点
def predict_from_excel():
        # 从 Excel 中读取数据
        wb = load_workbook(
                r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程\Intelligent-temperature-and-humidity-system\public\mqtt_data.xlsx")
        ws = wb.active
        data = pd.DataFrame(ws.values)
        data.columns = ["Temperature", "Humidity", "deviceId", "Timestamp"]    # 设置列名

```

```
71.         #  忽略表头行
72.         data  =  data.iloc[1:]
73.
74.         #  进行数据预处理
75.         data_values  =  data[["Temperature",  "Humidity"]].astype(float).
    values    #  转换为浮点数
76.         scaled_data  =  scaler.fit_transform(data_values)
77.
78.         #  创建时间序列数据集
79.         time_step  =  10    #  修改为 10 步进行试验
80.         X  =  []
81.         for  i  in  range(len(scaled_data)  -  time_step):
82.                 X.append(scaled_data[i:i  +  time_step])
83.         X  =  np.array(X)
84.
85.         #  转换为  LSTM  模型所需的输入形状
86.         current_input  =  torch.tensor(X[-1].reshape(1,  time_step,  2),
      dtype=torch.float32)
87.
88.         predictions  =  []
89.         future_times  =  []
90.
91.         #  获取当前时间
92.         current_time  =  datetime.now()
93.
94.         for  i  in  range(60):    #  预测未来  60  个点
95.                 with  torch.no_grad():
96.                         prediction  =  model(current_input)    #  预测下
    一个时间步
97.
98.                 #  逆归一化
99.                 predicted_values  =  scaler.inverse_transform(prediction.
    numpy())
100.
101.                 #  保存预测结果
102.                 predictions.append(predicted_values[0])
103.
104.                 #  计算预测时间
105.                 future_time  =  current_time  +  timedelta(minutes=(i
      +  1))    #  每隔  1  分钟递增
106.                 future_times.append(future_time.strftime('%H:%M:%S'))
107.
108.                 #  使用预测结果作为下一个时间步的输入
109.                 current_input  =  torch.tensor(
```

```
                        np.append(current_input[:, 1:, :], predicti
    on.unsqueeze(1), axis=1), dtype=torch.float32
                    )

            # 将预测结果覆盖写入 Excel 文件
            wb = Workbook()    # 创建新的工作簿
            ws = wb.active
            ws.title = "Predicted Data"
            ws.append(["Temperature", "Humidity", "deviceId", "Timestam
    p"])    # 设置表头

            for i, (temp, hum) in enumerate(predictions):
                    # 写入数据到工作表，ID 为 "Predicted"，时间为未来预
    测时间
                    ws.append([temp, hum, "1", future_times[i]])

            # 保存预测数据到 Excel 文件
            wb.save(
                    r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
    \Intelligent-temperature-and-humidity-system\public\predicted_data.xlsx"
    )


    # 当客户端接收到一个 CONNACK 响应时的回调函数
    def on_connect(client, userdata, flags, rc):
            print("Connected with result code " + str(rc))
            client.subscribe(topic)

    # 当从服务器中收到消息时的回调函数
    def on_message(client, userdata, msg):
            try:
                    # 解析消息内容
                    payload = msg.payload.decode("utf-8")
                    data = json.loads(payload)

                    # 从数据中提取字段
                    temperature = data.get("Temperature")
                    humidity = data.get("Humidity")
                    sensor_id = data.get("ID")
                    timestamp = data.get("Timestamp")

                    # 将时间戳转化为时分秒格式
                    readable_time = time.strftime('%H:%M:%S', time.loca
    ltime(timestamp))
```

```
148.
149.                 #  打印接收到的数据
150.                 print(
151.                         f"Received  data:  Temperature={temperature},
    Humidity={humidity},  ID={sensor_id},  Timestamp={readable_time}"
152.                 )
153.
154.                 #  将数据写入  Excel（mqtt_data.xlsx）
155.                 wb  =  load_workbook(
156.                         r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
  \Intelligent-temperature-and-humidity-system\public\mqtt_data.xlsx")
  #  打开已存在的  Excel  文件
157.                 ws  =  wb.active
158.                 ws.append([temperature,  humidity,  sensor_id,  readab
  le_time])    #  追加数据
159.                 wb.save(
160.                         r"C:\Users\asus\Desktop\x\试卷卷卷子\网络工程
  \Intelligent-temperature-and-humidity-system\public\mqtt_data.xlsx")
161.
162.                 #  每次新数据到达后从  Excel  读取并进行预测
163.                 predict_from_excel()
164.
165.         except  Exception  as  e:
166.                 print(f"Error  processing  message:  {e}")
167.
168. #  构建  MQTT  客户端
169. client  =  mqtt.Client()
170.
171. #  配置回调函数
172. client.on_connect  =  on_connect
173. client.on_message  =  on_message
174.
175. #  连接服务器
176. client.connect(host,  port)
177.
178. #  启动  MQTT  客户端并开始循环
179. client.loop_forever()
180.
```

创建 Excel 文件：

- 脚本会创建两个 Excel 文件，一个用于存储原始数据（mqtt_data.xlsx），另一个用于存储预测数据（predicted_data.xlsx）。

LSTM 模型定义：

- LSTM 模型定义：脚本使用一个双向 LSTM 模型来预测未来的温湿度值。模型通过加载 model.pth 文件中的预训练权重来初始化。

预测功能：

- 当收到新的数据时，脚本会从 mqtt_data.xlsx 中读取温湿度数据并进行归一化处理。随后，数据被送入 LSTM 模型进行未来 60 个时间步（分钟）的预测，结果写入 predicted_data.xlsx 文件。

MQTT 客户端：

- 通过 MQTT 协议连接服务器，订阅指定的主题（esp8266），接收到消息后调用 on_message 回调函数，进行数据处理和预测。

4.5.4 后端主函数代码

```
1. from flask import Flask, render_template, request, jsonify,
    redirect, url_for, flash
2. from extension import db
3. from models import Students, Teachers, Courses, Reports
4. from datetime import datetime
5. import secrets
6. from flask_cors import CORS
7. from api.courses import courses_api
8. from api.reports import reports_api
9. from api.students import students_api
10.from api.teachers import teachers_api
11.from api.realities import realities_api
12.from api.predictions import predictions_api
13.
14.app = Flask(__name__, static_folder='unimp')
15.# 配置数据库的地址
16.app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:admin
   123456@localhost:3306/database'
17.# 跟踪数据库的修改 --> 不建议开启 未来的版本中会移除
18.app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
19.app.config['SECRET_KEY'] = secrets.token_hex(16)
20.# 启用 CORS，并允许所有来源的请求访问你的 Flask 应用中的所有路径
21.
22.# 启用 CORS 并允许携带凭据
23.CORS(app, supports_credentials=True)
```

```
24.
25.# 数据库初始化
26.db.init_app(app)
27.
28.# 注册各个 API 蓝图
29.app.register_blueprint(courses_api)
30.app.register_blueprint(reports_api)
31.app.register_blueprint(students_api)
32.app.register_blueprint(teachers_api)
33.app.register_blueprint(realities_api)
34.app.register_blueprint(predictions_api)
35.
36.
37.if __name__ == "__main__":
38.    with app.app_context():
39.        db.create_all()  # 创建所有表
40.    app.run(debug=True)
```

4.5.5 相应后端 API 代码（部分）

```
1. # predictions.py
2. from flask import Blueprint, jsonify, request
3. from models import Predictions, db
4. import pandas as pd
5.
6. predictions_api = Blueprint('predictions_api', __name__)
7.
8. @predictions_api.route('/predictions/predictions', methods=
   ['GET'])
9. def get_predictions():
10.    # 从数据库获取最新的前 10 条预测数据
11.    try:
12.        data = Predictions.query.order_by(Predictions.NO.de
   sc()).limit(10).all()
13.        return jsonify([item.to_dict() for item in data])
14.    except Exception as e:
15.        predictions_api.logger.error(f"Error fetching predi
   ctions: {str(e)}")
16.        return jsonify({'error': str(e)}), 500
17.
18.@predictions_api.route('/predictions/data', methods=['GET']
   )
```

```
def get_history_predictions():
    # 从数据库获取所有预测数据
    try:
        data = Predictions.query.all()
        return jsonify([item.to_dict() for item in data])
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@predictions_api.route('/predictions/upload', methods=['POST'])
def upload_predictions():
    # 处理文件上传并存入预测数据到数据库
    if 'file' not in request.files:
        return jsonify({'error': '没有文件上传'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': '没有选择文件'}), 400

    if file and file.filename.endswith(('.xlsx', '.xls')):
        try:
            data = pd.read_excel(file)
            # 假设数据包含必要的字段，如 Temperature, Humidity, DeviceId, Timestamp 等
            for index, row in data.iterrows():
                new_prediction = Predictions(
                    Temperature=row['Temperature'],
                    Humidity=row['Humidity'],
                    DeviceId=row['DeviceId'],
                    Timestamp=row['Timestamp'],
                )
                db.session.add(new_prediction)
            db.session.commit()
            return jsonify({'message': '文件上传成功'}), 200
        except Exception as e:
            db.session.rollback()
            return jsonify({'error': str(e)}), 500
    else:
        return jsonify({'error': '文件格式不支持'}), 400



from flask import Blueprint, jsonify, request
from models import Realities, db
```

```
import pandas as pd

realities_api = Blueprint('realities_api', __name__)

@realities_api.route('/realities/realities', methods=['GET'])
def get_realities():
    # 从数据库获取最新的前10条数据
    try:
        data = Realities.query.order_by(Realities.id.desc()).limit(10).all()
        return jsonify([item.to_dict() for item in data])
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@realities_api.route('/realities/data', methods=['GET'])
def get_history_data():
    # 从数据库获取历史数据（假设有一个字段来区分历史数据）
    try:
        data = Realities.query.all()
        return jsonify([item.to_dict() for item in data])
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@realities_api.route('/realities/upload', methods=['POST'])
def upload_data():
    # 处理文件上传并存入数据库
    if 'file' not in request.files:
        return jsonify({'error': '没有文件上传'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': '没有选择文件'}), 400

    if file and file.filename.endswith(('.xlsx', '.xls')):
        try:
            data = pd.read_excel(file)
            # 假设数据包含必要的字段，如name, description等
            for index, row in data.iterrows():
                new_reality = Realities(
                    name=row['name'],
                    description=row['description'],
                    # 添加其他字段
                )
```

```
                db.session.add(new_reality)
            db.session.commit()
            return jsonify({'message': '文件上传成功
'}), 200
        except Exception as e:
            db.session.rollback()
            return jsonify({'error': str(e)}), 500
    else:
        return jsonify({'error': '文件格式不支持'}), 400

@realities_api.route('/realities/temperature', methods=['
GET'])
def get_temperature_data():
    try:
        # 获取数据库中的温度数据并按时间排序
        data = Realities.query.with_entities(Realities.Ti
mestamp, Realities.Temperature).all()
        # 转换为 JSON 格式
        result = [{"time": item.Timestamp.strftime('%H:%M:
%S'), "temperature": item.Temperature} for item in data]
        return jsonify(result), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500


@realities_api.route('/realities/humidity', methods=['GET
'])
def get_humidity_data():
    try:
        # 获取数据库中的湿度数据并按时间排序
        data = Realities.query.with_entities(Realities.Ti
mestamp, Realities.Humidity).all()
        # 转换为 JSON 格式
        result = [{"time": item.Timestamp.strftime('%H:%M:
%S'), "humidity": item.Humidity} for item in data]
        return jsonify(result), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500


```

4.5.6 前端主界面

```
<template>
  <v-sheet :elevation="2" :height="205" :width="350" class="rounded-lg"
    style="position: absolute; top: 50px; left: 100px;">
    <div class="container">
      <div class="title text-subtitle-2">室外气象环境监控</div>
      <div class="timestamp text-subtitle-2">更新时间: 16:50</div>
    </div>

    <v-divider class="border-opacity-25"></v-divider>

    <v-table class="text-caption" density="compact">
      <tbody>
        <tr align="right" style="white-space: nowrap;">
          <th>温度:</th>
          <td>14℃</td>
        </tr>

        <tr align="right">
          <th>湿度:</th>
          <td>53%</td>
        </tr>

        <tr align="right">
          <th>太阳光照:</th>
          <td>3842LUX</td>
        </tr>

        <tr align="right">
          <th>太阳辐射</th>
          <td>8.7W/m2</td>
        </tr>
      </tbody>
    </v-table>
  </v-sheet>

  <v-sheet :elevation="2" :height="180" :width="350" class="rounded-lg"
    style="position: absolute; top: 350px; left: 100px;">
    <div class="container">
```

```
39.       <div class="title text-subtitle-2">三日天气预报</div>
40.     </div>
41.
42.     <v-divider class="border-opacity-25"></v-divider>
43.
44.     <v-row>
45.       <v-col col="auto" class="mx-auto text-center">
46.         <div class="text-h6">周三</div>
47.
48.         <v-icon color="black" icon="mdi-weather-pouring" si
    ze="60" class="py-lg-4"></v-icon>
49.
50.         <div class="py-lg-1">11℃~17℃</div>
51.       </v-col>
52.
53.       <v-divider vertical style="margin-top: 12px;"></v-div
    ider>
54.
55.       <v-col col="auto" class="mx-auto text-center">
56.         <div class="text-h6">周四</div>
57.
58.         <v-icon color="error" icon="mdi-weather-lightning"
    size="60" class="py-lg-4"></v-icon>
59.
60.         <div class="py-lg-1">9℃~14℃</div>
61.       </v-col>
62.
63.       <v-divider vertical style="margin-top: 12px;"></v-div
    ider>
64.
65.       <v-col col="auto" class="mx-auto text-center">
66.         <div class="text-h6">周五</div>
67.
68.         <v-icon color="black" icon="mdi-weather-sunny" size
    ="60" class="py-lg-4"></v-icon>
69.
70.         <div class="py-lg-1">13℃~20℃</div>
71.       </v-col>
72.     </v-row>
73.
74.   </v-sheet>
75.
76.   <v-sheet :elevation="2" :height="40" :width="350" class="
    rounded-lg"
```

```
77.     style="position: absolute; top: 600px; left: 100px;">
78.     <div class="container">
79.       <div class="title text-subtitle-2">实时视频监控</div>
80.     </div>
81.   </v-sheet>
82.
83.   <v-sheet :elevation="2" :height="220" :width="350" class=
   "rounded-lg"
84.     style="position: absolute; top: 630px; left: 100px;">
85.     <div class="container">
86.       <!-- <div class="title text-h4">这里放视频</div> -->
87.       <video src="@/assets/video.mp4" controls width="350"
   height="200" autoplay></video>
88.
89.
90.     </div>
91.   </v-sheet>
92.
93.   <v-sheet :elevation="2" :height="108" :width="700" class=
   "rounded-lg"
94.     style="position: absolute; top: 50px; left: 600px;">
95.     <div class="container">
96.       <div class="title text-subtitle-2">温室环境监控</div>
97.     </div>
98.
99.     <v-divider class="border-opacity-25"></v-divider>
100.
101.      <v-row class="py-lg-2">
102.        <v-col col="auto" class="mx-auto text-center">
103.          <div>空气温度</div>
104.          <div>24℃</div>
105.        </v-col>
106.
107.        <v-divider vertical style="margin-top: 4px;"></v-di
   vider>
108.
109.        <v-col col="auto" class="mx-auto text-center">
110.          <div>空气湿度</div>
111.          <div>64%</div>
112.        </v-col>
113.
114.        <v-divider vertical style="margin-top: 4px;"></v-di
   vider>
115.
```

```
116.         <v-col col="auto" class="mx-auto text-center">
117.           <div>CO2 浓度</div>
118.           <div>843 ppm</div>
119.         </v-col>
120.
121.         <v-divider vertical style="margin-top: 4px;"></v-di
   vider>
122.
123.         <v-col col="auto" class="mx-auto text-center">
124.           <div>光照强度</div>
125.           <div>1039 lux</div>
126.         </v-col>
127.       </v-row>
128.
129.     </v-sheet>
130.
131.     <div id="threejs" style="position: absolute; top: 200px;
   left: 500px"></div>
132.
133.     <v-sheet :elevation="2" :height="auto" :width="250" cla
   ss="rounded-lg"
134.       style="position: absolute; top: 50px; left: 1600px;">
135.     <div class="container">
136.       <div class="title text-subtitle-2">温室设备控制</div>
137.     </div>
138.
139.     <v-divider class="border-opacity-25"></v-divider>
140.
141.     <!-- 修改 v-row 为垂直方向 -->
142.     <v-row class="py-lg-4" direction="column" align="center
   ">
143.
144.       <!-- 风机设备控制 -->
145.       <v-col col="auto" class="mx-auto text-center">
146.         <img src="@/assets/fan.png" alt="Icon" style="width:
    30px; height: 30px;">
147.         <div class="py-lg-1">风机</div>
148.         <v-switch color="primary" label="开关
   " class="d-flex justify-center"></v-switch>
149.       </v-col>
150.
151.       <v-divider style="margin-top: 4px;"></v-divider>
152.
153.       <!-- 照明设备控制 -->
```

```
154.       <v-col col="auto" class="mx-auto text-center">
155.         <img src="@/assets/light.png" alt="Icon" style="wid
   th: 30px; height: 30px;">
156.         <div class="py-lg-1">照明</div>
157.         <v-switch color="primary" label="开关
   " class="d-flex justify-center"></v-switch>
158.       </v-col>
159.
160.       <v-divider style="margin-top: 4px;"></v-divider>
161.
162.       <!-- 另一照明设备控制 -->
163.       <v-col col="auto" class="mx-auto text-center">
164.         <img src="@/assets/light.png" alt="Icon" style="wid
   th: 30px; height: 30px;">
165.         <div class="py-lg-1">照明</div>
166.         <v-switch color="primary" label="开关
   " class="d-flex justify-center"></v-switch>
167.       </v-col>
168.
169.       <v-divider style="margin-top: 4px;"></v-divider>
170.
171.       <!-- 另一风机设备控制 -->
172.       <v-col col="auto" class="mx-auto text-center">
173.         <img src="@/assets/fan.png" alt="Icon" style="width:
    30px; height: 30px;">
174.         <div class="py-lg-1">风机</div>
175.         <v-switch color="primary" label="开关
   " class="d-flex justify-center"></v-switch>
176.       </v-col>
177.
178.     </v-row>
179.   </v-sheet>
180.
181.   </template>
182.
183.   <script setup>
184.   // import InfoCard from '@/components/index/InfoCard.vue'
   ;
185.   // import MapContainer from '@/components/index/MapContai
   ner.vue';
186.   // import DataPanel from '@/components/index/DataPanel.vu
   e';
187.   import { ref } from 'vue';
188.   import axiosRequest from "@/utils/request";
```

```
189.  import { onMounted } from 'vue';
190.  import * as echarts from 'echarts';
191.  import { nextTick } from 'vue';
192.  import * as THREE from 'three';
193.  import { OrbitControls } from 'three/addons/controls/Orbi
   tControls.js';
194.  import { GLTFLoader } from 'three/addons/loaders/GLTFLoad
   er.js';
195.  import { RoomEnvironment } from 'three/addons/environment
   s/RoomEnvironment.js';
196.
197.  // 从后端获取到数据以后在前端进行展示
198.  const _isRequestSuccess = ref(false);
199.  const _isLoading = ref(false);
200.  const _selectDeviceId = ref(-1);
201.  const _latestData = ref({});
202.  const _geoAddress = ref({});
203.
204.  const model = ref("no")
205.
206.  // 传递给子组件
207.  // let DataPanelProps = {
208.  //   _isRequestSuccess,
209.  //   _isLoading,
210.  //   _selectDeviceId,
211.  //   _latestData,
212.  //   _geoAddress
213.  // }
214.
215.  // function handleChange() {
216.  //   if (model.value == 'no') {
217.  //     model.value = 'yes'
218.  //   } else {
219.  //     model.value = 'no'
220.  //   }
221.  // }
222.
223.  // 点击到地图上的坐标以后发起后端请求
224.  // function pointClickEvent(id) {
225.  //   _isRequestSuccess.value = false;
226.  //   _isLoading.value = true;
227.  //   _selectDeviceId.value = id;
228.
```

```
229.  //   let promise_latestDataById = new Promise((resolve) =
   > {
230.  //     axiosRequest.get("/api/device/latestDataById", {
231.  //       params: {
232.  //         id
233.  //       }
234.  //     })
235.  //       .then(res => {
236.  //         _latestData.value = res;
237.  //         resolve();
238.  //       });
239.  //   });
240.
241.  //   let promise_deviceGeoAddressById = new Promise((reso
   lve) => {
242.  //     axiosRequest.get("/api/device/deviceGeoAddressById
   ", {
243.  //       params: {
244.  //         id
245.  //       }
246.  //     })
247.  //       .then(res => {
248.  //         _geoAddress.value = res;
249.  //         resolve();
250.  //       });
251.  //   });
252.
253.  //   Promise.all([promise_latestDataById, promise_deviceG
   eoAddressById])
254.  //     .then(() => {
255.  //       _isLoading.value = false;
256.  //       _isRequestSuccess.value = true;
257.  //     });
258.  // }
259.
260.  onMounted(() => {
261.
262.    // 透视摄像头
263.    const camera = new THREE.PerspectiveCamera(75, window.i
   nnerWidth / window.innerHeight, 0.1, 1000);
264.    camera.position.z = 5;
265.
266.    // 基础渲染器
```

```
267.    // const renderer = new THREE.WebGLRenderer({ alpha: tr
    ue });
268.    const renderer = new THREE.WebGLRenderer({ alpha: true }
    );
269.    renderer.setPixelRatio(window.devicePixelRatio);
270.    renderer.setSize(1000, 700);
271.    renderer.setClearColor(0xffffff);
272.    document.getElementById('threejs').appendChild(renderer.
    domElement);
273.
274.    // 可移动的摄像头
275.    const controls = new OrbitControls(camera, renderer.dom
    Element);
276.    controls.target.set(0, 0.5, 0);
277.    controls.update();
278.    // controls.enablePan = false;
279.    controls.enableDamping = true;
280.
281.    // 辐射场景贴图
282.    const pmremGenerator = new THREE.PMREMGenerator(rendere
    r);
283.
284.    // const geometry = new THREE.BoxGeometry(1, 1, 1);
285.    // const material = new THREE.MeshBasicMaterial({ color:
     0x00ff00 });
286.    // const cube = new THREE.Mesh(geometry, material);
287.    // scene.add(cube);
288.
289.    // 创建场景
290.    const scene = new THREE.Scene();
291.    scene.environment = pmremGenerator.fromScene(new RoomEn
    vironment(renderer), 0.04).texture;
292.
293.    // 模型加载器
294.    const loader = new GLTFLoader();
295.    loader.load('src/pages/garden_greenhouse/scene.gltf', f
    unction (gltf) {
296.
297.      scene.add(gltf.scene);
298.
299.    }, undefined, function (error) {
300.
301.      console.error(error);
302.
```

```
303.     });
304.
305.     function animate() {
306.       requestAnimationFrame(animate);
307.       controls.update();
308.       // cube.rotation.x += 0.01;
309.       // cube.rotation.y += 0.01;
310.       renderer.render(scene, camera);
311.     }
312.     animate();
313.   })
314.   </script>
315.   <style>
316.   .container {
317.     display: flex;
318.     width: 100%;
319.   }
320.
321.   .title {
322.     flex-grow: 1;
323.     /* font-size: 2em; */
324.     /* 大标题字体大小 */
325.     padding: 10px;
326.     font-family: Arial, sans-serif;
327.     /* 使用Arial 字体 */
328.     box-sizing: border-box;
329.     /* 防止padding 影响最终尺寸 */
330.   }
331.
332.   .timestamp {
333.     /* font-size: 1em; */
334.     /* 时间戳字体大小 */
335.     padding: 10px;
336.     font-family: Arial, sans-serif;
337.     box-sizing: border-box;
338.     white-space: nowrap;
339.     /* 防止时间戳折行 */
340.   }
341.   </style>
```

4.5.7 前端 3D 模型展示页面

```
<template>
  <div id="threejs-container">
  </div>
  <!-- 终端#1 -->
  <v-sheet id="threejs-tag-1" class="threejs-label" width="6%">
    <v-row class="py-lg-1 px-lg-2">
      <div class="mx-auto text-center">农田设备#1</div>
    </v-row>

    <v-row>
      <v-col col="auto" class="mx-auto text-center text-body-2" @click.stop="getRealTimeData(1)">
        <div>实时数据</div>
      </v-col>

      <v-col col="auto" class="mx-auto text-center text-body-2" @click.stop="showHistoryDataDialog(1)">
        <div>历史数据</div>
      </v-col>
    </v-row>

    <v-row class="align-center">
      <v-switch v-model="device1_Model" false-value="no" true-value="yes" hide-details class="px-lg-3"></v-switch>
      <div :class="device1_ClassComputed">水泵:{{ device1_ModelComputed }}</div>
    </v-row>
  </v-sheet>

  <!-- 终端#2 -->
  <v-sheet id="threejs-tag-2" class="threejs-label" width="6%">
    <v-row class="py-lg-1 px-lg-2">
      <div class="mx-auto text-center">农田设备#2</div>
    </v-row>

    <v-row>
      <v-col col="auto" class="mx-auto text-center text-body-2" @click.stop="getRealTimeData(2)">
        <div>实时数据</div>
      </v-col>
```

```
36.
37.       <v-col col="auto" class="mx-auto text-center text-bod
   y-2" @click.stop="showHistoryDataDialog(2)">
38.         <div>历史数据</div>
39.       </v-col>
40.     </v-row>
41.
42.     <v-row class="align-center">
43.       <v-switch v-model="device2_Model" false-value="no" tr
   ue-value="yes" hide-details></v-switch>
44.       <div :class="device2_ClassComputed">水
   泵:{{ device2_ModelComputed }}</div>
45.     </v-row>
46.   </v-sheet>
47.   <!-- 空调标签组 -->
48.   <!-- <div id="threejs-tag-5" class="threejs-label">空调
   </div> -->
49.   <v-sheet id="threejs-tag-5" class="threejs-label" width="
   5%">
50.     <v-row class="py-lg-1 px-lg-2">
51.       <div class="mx-auto text-center">空调</div>
52.     </v-row>
53.
54.     <v-row class="align-center">
55.       <v-switch v-model="airConditionModel" false-value="no
   " true-value="yes" hide-details></v-switch>
56.       <div :class="airConditionClassComputed">{{ airConditi
   onModelComputed }}</div>
57.     </v-row>
58.   </v-sheet>
59.
60.   <!-- 监控设备的标签 -->
61.   <v-sheet id="threejs-tag-6" class="threejs-label" width="
   7%">
62.     <v-row>
63.       <div class="mx-auto text-center">监控设备</div>
64.     </v-row>
65.
66.     <v-row>
67.       <v-col col="auto" class="text-center" @click.stop="ge
   tRealTimeVideoUri">
68.         <div>实时监控</div>
69.       </v-col>
70.
```

```
71.       <v-col col="auto" class="text-center" @click.stop="ge
   tPlayBackVideoUri">
72.         <div>视频回放</div>
73.       </v-col>
74.     </v-row>
75.   </v-sheet>
76.
77.   <!-- 电风扇标签组 -->
78.   <v-sheet id="threejs-tag-7" class="threejs-label" width="
   5%">
79.     <v-row class="py-lg-1 px-lg-2">
80.       <div class="mx-auto text-center">风机</div>
81.     </v-row>
82.
83.     <v-row class="align-center">
84.       <v-switch v-model="fanModel" false-value="no" true-va
   lue="yes" hide-details></v-switch>
85.       <div :class="fanClassComputed">{{ fanModelComputed }}
   </div>
86.     </v-row>
87.   </v-sheet>
88.
89.   <!-- 照明灯标签组 -->
90.   <!-- <div id="threejs-tag-8" class="threejs-label">照明灯
   </div> -->
91.   <v-sheet id="threejs-tag-8" class="threejs-label" width="
   5%">
92.     <v-row class="py-lg-1 px-lg-2">
93.       <div class="mx-auto text-center">照明灯</div>
94.     </v-row>
95.
96.     <v-row class="align-center">
97.       <v-switch v-model="lightModel" false-value="no" true-
   value="yes" hide-details></v-switch>
98.       <div :class="lightClassComputed">{{ lightModelCompute
   d }}</div>
99.     </v-row>
100.    </v-sheet>
101.
102.    <!-- 温室大棚室外数据 -->
103.    <v-sheet class="transparent-background" width="38vh" st
   yle="position: absolute; top: 65vh; left: 10vh;z-index:2">
104.      <div class="gradient-background">
```

```
105.        <div class="gradient-text text-title text-base-styl
    e text-very-big-font">室外环境数据</div>
106.      </div>
107.
108.      <!-- <v-divider class="border-opacity-25"></v-divider
    > -->
109.
110.      <v-row>
111.        <v-col>
112.          <v-table class="transparent">
113.            <div class="text-base-style text-middle-font">
    空气温度</div>
114.            <div class="text-base-style text-big-font blue-
    text">21.5℃</div>
115.          </v-table>
116.        </v-col>
117.
118.        <v-col>
119.          <v-table class="transparent">
120.            <div class="text-base-style text-middle-font">
    空气湿度</div>
121.            <div class="text-base-style text-big-font blue-
    text">54.42%</div>
122.          </v-table>
123.        </v-col>
124.      </v-row>
125.
126.      <v-row>
127.        <v-col>
128.          <v-table class="transparent">
129.            <div class="text-base-style text-middle-font">
    太阳光照</div>
130.            <div class="text-base-style text-big-font blue-
    text">178LUX</div>
131.          </v-table>
132.        </v-col>
133.
134.        <v-col>
135.          <v-table class="transparent">
136.            <div class="text-base-style text-middle-font">
    太阳辐射</div>
137.            <div class="text-base-style text-big-font blue-
    text">8.7W/m2</div>
138.          </v-table>
```

```
139.         </v-col>
140.       </v-row>
141.     </v-sheet>
142.
143.     <!-- 设备实时、历史数据标签组 -->
144.     <v-sheet width="38vh" style="position: absolute; top: 1
    vh; left:10vh;height: 5vh;z-index:2" class="gradient-backgr
    ound">
145.       <div class="gradient-text text-title text-base-style
    text-very-big-font">植物生长分析</div>
146.     </v-sheet>
147.
148.     <v-sheet width="38vh" v-if="showlatestData == false" st
    yle="position: absolute; top: 7vh; left:10vh;height: 10vh;z
    -index:2"
149.       class="gradient-background">
150.       <div class="gradient-text text-title text-base-style
    text-very-big-font" style="height: 10vh;">选择农田设备查看
    </div>
151.     </v-sheet>
152.
153.     <v-sheet width="37vh" v-if="showlatestData == true" sty
    le="position: absolute; top: 7vh; left:10vh;height: auto "
154.       class="transparent-background">
155.       <v-row v-if="selectedDevice == 1">
156.         <v-col>
157.           <img src="./1.jpg" width="250vh">
158.         </v-col>
159.
160.         <v-col>
161.           <v-row class="container-space-between blue-text t
    ext-base-style text-big-font">
162.             <div>植物名:</div>
163.             <div>光瓜栗</div>
164.           </v-row>
165.
166.           <v-row class="container-space-between blue-text t
    ext-base-style text-big-font">
167.             <div>生长阶段：</div>
168.             <div>幼苗</div>
169.           </v-row>
170.
171.           <v-row class="container-space-between blue-text t
    ext-base-style text-big-font">
```

```
172.             <div>生长状况：</div>
173.             <div>良好</div>
174.           </v-row>
175.
176.           <v-row class="container-space-between blue-text t
   ext-base-style text-big-font">
177.             <div>病虫害：</div>
178.             <div>无</div>
179.           </v-row>
180.         </v-col>
181.       </v-row>
182.
183.       <v-row v-if="selectedDevice == 2">
184.         <v-col>
185.           <img src="./2.jpg" width="200vh">
186.         </v-col>
187.
188.         <v-col>
189.           <v-row class="container-space-between blue-text t
   ext-base-style text-big-font">
190.             <div>植物名:</div>
191.             <div>光瓜栗</div>
192.           </v-row>
193.
194.           <v-row class="container-space-between blue-text t
   ext-base-style text-big-font">
195.             <div>生长阶段：</div>
196.             <div>幼苗</div>
197.           </v-row>
198.
199.           <v-row class="container-space-between blue-text t
   ext-base-style text-big-font">
200.             <div>生长状况：</div>
201.             <div>土壤缺水</div>
202.           </v-row>
203.
204.           <v-row class="container-space-between blue-text t
   ext-base-style text-big-font">
205.             <div>病虫害：</div>
206.             <div>无</div>
207.           </v-row>
208.         </v-col>
209.       </v-row>
210.
```

```
211.      <v-row>
212.        <v-col>
213.          <v-table class="transparent">
214.            <div class="text-base-style text-middle-font">
    空气温度</div>
215.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.airTemperature }}℃</div>
216.          </v-table>
217.        </v-col>
218.
219.        <v-col>
220.          <v-table class="transparent">
221.            <div class="text-base-style text-middle-font">
    空气湿度</div>
222.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.airHumidity }}%</div>
223.          </v-table>
224.        </v-col>
225.      </v-row>
226.
227.      <v-row>
228.        <v-col>
229.          <v-table class="transparent">
230.            <div class="text-base-style text-middle-font">
    土壤温度</div>
231.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.soilTemperature }}℃</div>
232.          </v-table>
233.        </v-col>
234.
235.        <v-col>
236.          <v-table class="transparent">
237.            <div class="text-base-style text-middle-font">
    土壤湿度</div>
238.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.soilHumidity }}%</div>
239.          </v-table>
240.        </v-col>
241.      </v-row>
242.
243.      <v-row>
244.        <v-col>
245.          <v-table class="transparent">
```

```
246.            <div class="text-base-style text-middle-font">
    太阳光照:</div>
247.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.light }}LUX</div>
248.          </v-table>
249.        </v-col>
250.
251.        <v-col>
252.          <v-table class="transparent">
253.            <div class="text-base-style text-middle-font">
    二氧化碳浓度</div>
254.            <div class="text-base-style text-big-font blue-
    text">{{ latestData.co2 }}ppm</div>
255.          </v-table>
256.        </v-col>
257.      </v-row>
258.    </v-sheet>
259.
260.    <!-- 右上角的播放视频标签组 -->
261.    <v-sheet width="40vh" style="position: absolute; top: 1
    vh; right:2vh;height: auto;z-index:2" class="gradient-backg
    round">
262.      <div class="gradient-text text-title text-base-style
    text-very-big-font">视频监控</div>
263.    </v-sheet>
264.
265.    <v-sheet width="40vh" v-show="videoUri == ''" style="po
    sition: absolute; top: 7vh; right: 2vh;height: 10vh;z-index:
    2"
266.      class="gradient-background">
267.      <div class="gradient-text text-title text-base-style
    text-very-big-font" style="height: 10vh;">请选择监控设备查看
    </div>
268.    </v-sheet>
269.
270.    <v-sheet width="40vh" v-show="videoUri != ''" style="po
    sition: absolute; top: 7vh; right: 2vh;height: 60vh; "
271.      class="transparent-background">
272.      <!-- <div class="gradient-text text-big-font">1111111
    1111</div> -->
273.      <v-btn variant="text blue-text" style="font-size: 50p
    x;height:4vh" @click="videoFullScreen">全屏播放</v-btn>
274.      <v-btn variant="text blue-text" style="font-size: 50p
    x;height:4vh" @click="closeVideo">关闭监控</v-btn>
```

```
275.
276.      <iframe :src="videoUri" frameborder="0" style="width:
    100%;height: 100%;margin-top: 2vh;"></iframe>
277.    </v-sheet>
278.
279.    <!-- 弹窗组件，用于搜索及展示历史数据 -->
280.    <v-dialog v-model="showDialog" width="90%">
281.      <v-card>
282.        <v-card-text>
283.          <!-- 数据选择框 -->
284.          <v-select label="选择查看的数据
   " variant="underlined" :items="historyDataSelectList" v-mod
   el="historyDataSelectType">
285.          </v-select>
286.
287.          <!-- 时间选择框 -->
288.          <v-menu v-model="showMenu" :close-on-content-clic
   k="false" location="bottom" transition="scroll-y-transition
   "
289.            min-width="auto">
290.            <template v-slot:activator="{ props }">
291.              <v-text-field label="选择时间范围
   " readonly v-bind="props" variant="underlined"
292.                v-model="timeRangeComputed"></v-text-field>
293.            </template>
294.
295.            <DatePicker id="datepicker" v-model.range="time
   Range" mode="date" />
296.          </v-menu>
297.
298.          <v-btn block variant="text" @click="searchHistory
   Data">
299.            搜索
300.          </v-btn>
301.        </v-card-text>
302.
303.        <v-card-text width>
304.          <div id="echarts" style="width: 100%;height:800px;
   "></div>
305.        </v-card-text>
306.
307.        <template v-slot:actions>
308.          <v-btn class="ms-auto" text="退出
   " @click="showDialog = false"></v-btn>
```

```
309.         </template>
310.       </v-card>
311.     </v-dialog>
312.
313.     <!-- 视频监控全屏弹窗组件 -->
314.     <v-dialog v-model="showFullScreenDialog" width="180vh"
   height="130vh">
315.       <iframe :src="videoUri" frameborder="0" style="width:
    100%;height: 100%;"></iframe>
316.
317.       <template v-slot:actions>
318.         <v-btn class="ms-auto" text="退出
   " @click="showFullScreenDialog = false"></v-btn>
319.       </template>
320.     </v-dialog>
321.  </template>
322.
323.  <script setup>
324.
325.  import { DatePicker } from 'v-calendar';
326.  import 'v-calendar/style.css';
327.  import { onMounted, computed, watch,nextTick } from 'vue'
   ;
328.  import { ref } from 'vue';
329.  import * as THREE from 'three';
330.  import { OrbitControls } from 'three/addons/controls/Orbi
   tControls.js';
331.  import { GLTFLoader } from 'three/addons/loaders/GLTFLoad
   er.js';
332.  import { EXRLoader } from 'three/addons/loaders/EXRLoader.
   js';
333.  import { RoomEnvironment } from 'three/addons/environment
   s/RoomEnvironment.js';
334.  import { TransformControls } from 'three/examples/jsm/con
   trols/TransformControls.js';
335.  import { CSS3DRenderer, CSS3DSprite } from 'three/example
   s/jsm/renderers/CSS3DRenderer';
336.  import { EffectComposer } from 'three/addons/postprocessi
   ng/EffectComposer.js';
337.  import { RenderPass } from 'three/addons/postprocessing/R
   enderPass.js';
338.  import { OutlinePass } from 'three/addons/postprocessing/
   OutlinePass.js';
```

```
339.  import { OutputPass } from 'three/addons/postprocessing/O
   utputPass.js';
340.  import { FXAAShader } from 'three/addons/shaders/FXAAShad
   er.js';
341.  import { ShaderPass } from 'three/addons/postprocessing/S
   haderPass.js';
342.  import { SMAAPass } from 'three/addons/postprocessing/SMA
   APass.js';
343.  import { addModelAndLable, addModel } from '@/utils/util'
   ;
344.  import axiosRequest from '@/utils/request';
345.  import { axiosOnenetCommand, axiosOnenetPumpCommand } fro
   m '@/utils/request';
346.  import { _startTime, _endTime } from '@/utils/date'
347.  import * as echarts from 'echarts';
348.
349.  const videoUri = ref('');
350.  const latestData = ref({});
351.  // 左侧数据源要展示的类型，0：不展示，1：实时数据
352.  const showlatestData = ref(false);
353.  // 进行历史搜索时提供的数据列表
354.  const historyDataSelectList = ref([
355.    { title: '空气温度', value: 1 },
356.    { title: '空气湿度', value: 2 },
357.    { title: '土壤温度', value: 3 },
358.    { title: '土壤湿度', value: 4 },
359.    { title: '光照强度', value: 5 },
360.    { title: '二氧化碳浓度', value: 6 },
361.  ]);
362.  // 被选中的搜索数据
363.  const historyDataSelectType = ref(null);
364.  // 被选中的设备 id
365.  const historyDataSelectDeviceId = ref(-1);
366.  // 点击历史数据时会跳出一个弹窗
367.  const showDialog = ref(false);
368.  // 全屏视频弹窗
369.  const showFullScreenDialog = ref(false);
370.  // 用于时间选择组件的跳出
371.  const showMenu = ref(false);
372.  // 显示/选择时间范围
373.  const timeRange = ref({
374.    start: new Date(),
375.    end: new Date(),
376.  });
```

```
377.  // 被选中展示实时数据的设备编号,用于展示不同的数据
378.  const selectedDevice = ref(0);
379.  // 照明灯开关绑定值
380.  const lightModel = ref('no');
381.  // 显示在网页上的值
382.  const lightModelComputed = computed(() => {
383.    return lightModel.value == 'no' ? '关灯' : '开灯';
384.  })
385.  const lightClassComputed = computed(() => {
386.    return lightModel.value == 'no' ? 'gray-text' : 'blue-t
   ext';
387.  })
388.  // 风扇绑定值
389.  const fanModel = ref('no')
390.  const fanModelComputed = computed(() => {
391.    return fanModel.value == 'no' ? '关机' : '开机';
392.  })
393.  const fanClassComputed = computed(() => {
394.    return fanModel.value == 'no' ? 'gray-text' : 'blue-tex
   t';
395.  })
396.  // 空调绑定值
397.  const airConditionModel = ref('no')
398.  const airConditionModelComputed = computed(() => {
399.    return airConditionModel.value == 'no' ? '关机' : '开机
   ';
400.  })
401.  const airConditionClassComputed = computed(() => {
402.    return airConditionModel.value == 'no' ? 'gray-text' :
   'blue-text';
403.  })
404.  // 农田设备#1 绑定值
405.  const device1_Model = ref('no')
406.  const device1_ModelComputed = computed(() => {
407.    return device1_Model.value == 'no' ? '关机' : '开机';
408.  })
409.  const device1_ClassComputed = computed(() => {
410.    return device1_Model.value == 'no' ? 'gray-text' : 'blu
   e-text';
411.  })
412.  // 农田设备#2 绑定值
413.  const device2_Model = ref('no')
414.  const device2_ModelComputed = computed(() => {
415.    return device2_Model.value == 'no' ? '关机' : '开机';
```

```
416.  })
417.  const device2_ClassComputed = computed(() => {
418.    return device2_Model.value == 'no' ? 'gray-text' : 'blu
    e-text';
419.  })
420.
421.  // 动画混合器数组，在每个周期中都要循环更新里面的全部mixer
422.  let mixers = [];
423.  // 动画播放组，控制动画是否播放
424.  let actions = [];
425.  // actionid 常量
426.  let actionId_fan = 'fan';
427.  let actionId_airCondition = 'airCondition';
428.
429.  // 直接用watch 来代替点击事件控制设备的开关
430.  watch(lightModel, newValue => {
431.    if (newValue == 'no') {
432.      lightClose();
433.    }
434.    else {
435.      lightOpen();
436.    }
437.  })
438.
439.  watch(fanModel, newValue => {
440.    if (newValue == 'no') {
441.      actions[actionId_fan].stop();
442.      fanClose();
443.    }
444.    else {
445.      actions[actionId_fan].play();
446.      fanOpen();
447.    }
448.  })
449.
450.  watch(airConditionModel, newValue => {
451.    if (newValue == 'no') {
452.      actions[actionId_airCondition].stop();
453.      airConditionClose();
454.    }
455.    else {
456.      actions[actionId_airCondition].play();
457.      airConditionOpen();
458.    }
```

```
459.  })
460.
461.  watch(device1_Model, newValue => {
462.    if (newValue == 'no') {
463.      pump1_Close();
464.    }
465.    else {
466.      pump1_Open();
467.    }
468.  })
469.
470.  watch(device2_Model, newValue => {
471.    if (newValue == 'no') {
472.      pump2_Close();
473.    }
474.    else {
475.      pump2_Open();
476.    }
477.  })
478.
479.  // 获得实时视频地址
480.  function getRealTimeVideoUri() {
481.    axiosRequest.get('/api/video/realTime')
482.      .then(res => {
483.        videoUri.value = '';
484.        videoUri.value = res;
485.      });
486.  }
487.
488.  // 获得视频回放地址
489.  function getPlayBackVideoUri() {
490.    axiosRequest.get('/api/video/playback')
491.      .then(res => {
492.        videoUri.value = '';
493.        videoUri.value = res;
494.      });
495.  }
496.
497.  // 照明设备
498.  function lightOpen() {
499.    axiosOnenetCommand(0, 1);
500.  }
501.
502.  function lightClose() {
```

```
503.   axiosOnenetCommand(0, 0);
504. }
505.
506. // 空调设备
507. function airConditionOpen() {
508.   axiosOnenetCommand(1, 1);
509. }
510.
511. function airConditionClose() {
512.   axiosOnenetCommand(1, 0);
513. }
514.
515. // 风机设备
516. function fanOpen() {
517.   axiosOnenetCommand(2, 1);
518. }
519.
520. function fanClose() {
521.   axiosOnenetCommand(2, 0);
522. }
523.
524. function pump1_Open() {
525.   axiosOnenetPumpCommand(3, 1);
526. }
527.
528. function pump1_Close() {
529.   axiosOnenetPumpCommand(3, 0);
530. }
531.
532. function pump2_Open() {
533.   axiosOnenetPumpCommand(4, 1);
534. }
535.
536. function pump2_Close() {
537.   axiosOnenetPumpCommand(4, 0);
538. }
539.
540. // 获得设备的实时数据
541. function getRealTimeData(id) {
542.   axiosRequest.get('/api/device/latestDataById', {
543.     params: {
544.       id
545.     }
546.   })
```

```
547.      .then(res => {
548.        latestData.value = res;
549.        selectedDevice.value = id;
550.        showlatestData.value = true;
551.      });
552.  }
553.
554.  // 点击历史数据按钮以后会跳出一个弹窗
555.  // 跳出弹窗之前清空原先的搜索数据
556.  function showHistoryDataDialog(id) {
557.    historyDataSelectType.value == null
558.    historyDataSelectDeviceId.value = id;
559.    timeRange.value = {
560.      start: new Date(),
561.      end: new Date(),
562.    };
563.
564.    showDialog.value = true;
565.  }
566.  // 全屏播放视频
567.  function videoFullScreen() {
568.    showFullScreenDialog.value = true;
569.  }
570.  // 关闭视频流
571.  function closeVideo() {
572.    videoUri.value = '';
573.  }
574.  // 从服务器搜索历史数据
575.  function searchHistoryData() {
576.    if (historyDataSelectType.value == null) return;
577.
578.    axiosRequest.get('/api/device/historyData', {
579.      params: {
580.        id: historyDataSelectDeviceId.value,
581.        dataType: historyDataSelectType.value,
582.        startDate: _startTime(timeRange.value.start),
583.        endDate: _endTime(timeRange.value.end)
584.      }
585.    })
586.      .then(async (res) => {
587.
588.        let myChart = echarts.init(document.getElementById(
    'echarts'));
589.        let option;
```

```
590.         let date = [];
591.         let data = [];
592.
593.         for (let i = 0; i < res.length; i++) {
594.           let now = new Date(res[i].dateTime);
595.           date.push([now.getFullYear(), now.getMonth() + 1,
    now.getDate()].join('/') + " " + [now.getHours(), now.getM
   inutes(), now.getSeconds()].join(':'));
596.           data.push(res[i].value);
597.         }
598.         option = {
599.           tooltip: {
600.             trigger: 'axis',
601.             position: function (pt) {
602.               return [pt[0], '10%'];
603.             }
604.           },
605.           title: {
606.             // left: 'center',
607.             // text: 'Large Area Chart'
608.           },
609.           toolbox: {
610.             feature: {
611.               restore: {},
612.               // saveAsImage: {}
613.             }
614.           },
615.           xAxis: {
616.             type: 'category',
617.             boundaryGap: false,
618.             data: date
619.           },
620.           yAxis: {
621.             type: 'value',
622.             boundaryGap: [0, 0.001]
623.           },
624.           dataZoom: [
625.             {
626.               type: 'inside',
627.               start: 0,
628.               end: 10
629.             },
630.             {
631.               start: 0,
```

```
632.             end: 10
633.           },
634.           {
635.             show: true,
636.             yAxisIndex: 0,
637.             filterMode: 'empty',
638.             width: 30,
639.             height: '80%',
640.             showDataShadow: false,
641.             left: '93%'
642.           }
643.         ],
644.         series: [
645.           {
646.             name: historyDataSelectList.value[historyData
    SelectType.value].title,
647.             type: 'line',
648.             symbol: 'none',
649.             sampling: 'lttb',
650.             itemStyle: {
651.               color: 'rgb(255, 70, 131)'
652.             },
653.             data: data
654.           }
655.         ]
656.       };
657.
658.       option && myChart.setOption(option);
659.     })
660. }
661. /**
662.  * 模型是异步加载，使用 cb 的方式调用模型的动画
663.  *
664.  * @param actionId 使用 id 来区分不同的 action
665.  */
666. const animationCallBack = (gltf, actionId) => {
667.   if (gltf.animations && gltf.animations.length) {
668.     // 创建动画混合器并将其与模型关联
669.     let mixer = new THREE.AnimationMixer(gltf.scene);
670.     // 由 action 来控制动画的播放
671.     let action = mixer.clipAction(gltf.animations[0]);
672.     // action.play();
673.
674.     mixers.push(mixer);
```

```
675.       actions[actionId] = action;
676.     }
677.   }
678.   const timeRangeComputed = computed(() => {
679.     return timeRange.value.start.toLocaleDateString() + ' -
     ' + timeRange.value.end.toLocaleDateString();
680.   })
681.   onMounted(async () => {
682.     await nextTick(); // 确保 DOM 完全渲染
683.
684.     const container = document.getElementById("threejs-cont
   ainer");
685.
686.     if (!container) {
687.       console.error("Container element not found");
688.       return;
689.     }
690.
691.     const containerWidth = container.offsetWidth;
692.     const containerHeight = container.offsetHeight;
693.
694.     // 创建透视摄像头
695.     const camera = new THREE.PerspectiveCamera(75, containe
   rWidth / containerHeight, 0.001, 1000);
696.     camera.position.z = 5;
697.
698.     // 基础渲染器
699.     const renderer = new THREE.WebGLRenderer({ alpha: true,
     antialias: true });
700.     renderer.setPixelRatio(window.devicePixelRatio);
701.     renderer.setSize(containerWidth, containerHeight);
702.     renderer.setClearColor(0xffffff);
703.     container.appendChild(renderer.domElement);
704.
705.     // 让摄像头可以移动
706.     const controls = new OrbitControls(camera, renderer.dom
   Element);
707.     controls.minDistance = 0.000001;
708.     controls.target.set(0, 0.5, 0);
709.     controls.update();
710.     controls.enableDamping = true;
711.
712.     // 创建场景
713.     const scene = new THREE.Scene();
```

```
714.
715.    // 辐射场景贴图，生成环境光
716.    const pmremGenerator = new THREE.PMREMGenerator(rendere
   r);
717.    scene.environment = pmremGenerator.fromScene(new RoomEn
   vironment(renderer), 0.001).texture;
718.
719.    // 添加环境光
720.    const ambientLight = new THREE.AmbientLight(0xffffff, 0.
   1);
721.    scene.add(ambientLight);
722.
723.    // 添加点光源
724.    const directionalLight = new THREE.PointLight(0xffffff,
    1, 100);
725.    directionalLight.position.set(3.812, 1.53, -2.811);
726.    scene.add(directionalLight);
727.
728.    // 创建 CSS3DRenderer 用于渲染标签
729.    const css3Render = new CSS3DRenderer();
730.    css3Render.setSize(containerWidth, containerHeight);
731.    css3Render.domElement.style.position = "absolute";
732.    css3Render.domElement.style.top = "0px";
733.    css3Render.domElement.style.left = "0px";
734.    css3Render.domElement.style.pointerEvents = "none";
735.    container.appendChild(css3Render.domElement);
736.
737.    // 天空盒加载器
738.    const exrLoader = new EXRLoader();
739.    exrLoader.load("garden.exr", (texture) => {
740.      texture.mapping = THREE.EquirectangularReflectionMapp
   ing;
741.      scene.background = texture; // 将 EXR 纹理设为背景
742.    });
743.
744.    // 模型加载器，用于导入模型
745.    const loader = new GLTFLoader();
746.
747.    let position, scale, rotation, label_position, label_sc
   ale;
748.
749.    // Example: 导入温室模型
750.    position = { x: 3.796, y: -1.969, z: -2.474 };
751.    scale = { x: 1, y: 1, z: 1 };
```

```
752.    rotation = { x: 0, y: 0, z: 0 };
753.    addModel("greenhouse_5.glb", position, scale, rotation,
    loader, scene, TransformControls, camera, controls, render
   er);
754.
755.    position = { x: 2.599, y: -1.816, z: 0.805 };
756.    // 缩放倍率不能为0
757.    scale = { x: 0.7, y: 0.7, z: 0.7 };
758.    // 参数示例：Math.PI / 4
759.    rotation = { x: 0, y: 0, z: 0 };
760.    label_position = { x: 0, y: 0.1, z: 0 };
761.    label_scale = { x: 0.005, y: 0.005, z: 0.005 };
762.    addModelAndLable('iot_device/scene.gltf',
763.      position, scale, rotation,
764.      'threejs-tag-1',
765.      label_position, label_scale,
766.      loader,
767.      CSS3DSprite,
768.      new THREE.Group(), scene,
769.      TransformControls, camera, controls, renderer
770.    );
771.
772.    position = { x: 6.027, y: -0.695, z: 1.728 };
773.    // 缩放倍率不能为0
774.    scale = { x: 0.7, y: 0.7, z: 0.7 };
775.    // 参数示例：Math.PI / 4
776.    rotation = { x: 0, y: 0, z: 0 };
777.    label_position = { x: 1, y: 0, z: 0 };
778.    label_scale = { x: 0.005, y: 0.005, z: 0.005 };
779.    addModelAndLable('iot_device/scene.gltf',
780.      position, scale, rotation,
781.      'threejs-tag-2',
782.      label_position, label_scale,
783.      loader,
784.      CSS3DSprite,
785.      new THREE.Group(), scene,
786.      TransformControls, camera, controls, renderer
787.    );
788.
789.    // position = { x: 4.294, y: -1.375, z: -0.841 };
790.    // // 缩放倍率不能为0
791.    // scale = { x: 0.7, y: 0.7, z: 0.7 };
792.    // // 参数示例：Math.PI / 4
793.    // rotation = { x: 0, y: 0, z: 0 };
```

```
794.    // label_position = { x: 0, y: 0.1, z: 0 };
795.    // label_scale = { x: 0.01, y: 0.01, z: 0.01 };
796.    // addModelAndLable('iot_device/scene.gltf',
797.    //   position, scale, rotation,
798.    //   'threejs-tag-3',
799.    //   label_position, label_scale,
800.    //   loader,
801.    //   CSS3DSprite,
802.    //   new THREE.Group(), scene,
803.    //   TransformControls, camera, controls, renderer
804.    // );
805.
806.    // position = { x: 4.453, y: 0.920, z: -2.176 };
807.    // // 缩放倍率不能为0
808.    // scale = { x: 0.6, y: 0.6, z: 0.6 };
809.    // // 参数示例：Math.PI / 4
810.    // rotation = { x: 0, y: 0, z: 0 };
811.    // label_position = { x: 0, y: 0.1, z: 0 };
812.    // label_scale = { x: 0.01, y: 0.01, z: 0.01 };
813.    // addModelAndLable('iot_device/scene.gltf',
814.    //   position, scale, rotation,
815.    //   'threejs-tag-4',
816.    //   label_position, label_scale,
817.    //   loader,
818.    //   CSS3DSprite,
819.    //   new THREE.Group(), scene,
820.    //   TransformControls, camera, controls, renderer
821.    // );
822.
823.    // 导入空调模型
824.    // 空调模型放在温室墙壁上
825.    position = { x: 1.644, y: 2.112, z: -4.169 };
826.    // 缩放倍率不能为0
827.    scale = { x: 2, y: 2, z: 2 };
828.    // 参数示例：Math.PI / 4
829.    rotation = { x: 0, y: Math.PI / 2.4, z: 0 };
830.    label_position = { x: 0, y: 0.3, z: 0 };
831.    label_scale = { x: 0.003, y: 0.003, z: 0.003 };
832.    addModelAndLable('fan/scene.gltf',
833.      position, scale, rotation,
834.      'threejs-tag-5',
835.      label_position, label_scale,
836.      loader,
837.      CSS3DSprite,
```

```
838.      new THREE.Group(), scene,
839.      TransformControls, camera, controls, renderer,
840.      animationCallBack, actionId_airCondition
841.    );
842.
843.    // 导入监控模型
844.    // 监控模型放在温室角落
845.    position = { x: -1.741, y: 2.478, z: 9.898 };
846.    // 缩放倍率不能为0
847.    scale = { x: 0.9, y: 0.9, z: 0.9 };
848.    // 参数示例：Math.PI / 4
849.    rotation = { x: 0, y: Math.PI / 1.5, z: 0 };
850.    label_position = { x: 0, y: 1, z: 0 };
851.    label_scale = { x: 0.01, y: 0.01, z: 0.01 };
852.    addModelAndLable('camera/scene.gltf',
853.      position, scale, rotation,
854.      'threejs-tag-6',
855.      label_position, label_scale,
856.      loader,
857.      CSS3DSprite,
858.      new THREE.Group(), scene,
859.      TransformControls, camera, controls, renderer
860.    );
861.
862.    // 导入风扇模型
863.    // 风扇模型放在温室门口
864.    position = { x: 8.147, y: 0.858, z: 4.502 };
865.    // 缩放倍率不能为0
866.    scale = { x: 2, y: 2, z: 2 };
867.    // 参数示例：Math.PI / 4
868.    rotation = { x: 0, y: Math.PI / 2.4, z: 0 };
869.    label_position = { x: 0.6, y: 0.15, z: 0 };
870.    label_scale = { x: 0.003, y: 0.003, z: 0.003 };
871.    addModelAndLable('fan/scene.gltf',
872.      position, scale, rotation,
873.      'threejs-tag-7',
874.      label_position, label_scale,
875.      loader,
876.      CSS3DSprite,
877.      new THREE.Group(), scene,
878.      TransformControls, camera, controls, renderer,
879.      animationCallBack, actionId_fan
880.    );
881.
```

```
882.    // 导入吊灯模型
883.    // 吊灯模型放在温室天花板
884.    position = { x: 5.444, y: 4.576, z: -4.373 };
885.    // 缩放倍率不能为0
886.    scale = { x: 0.15, y: 0.15, z: 0.15 };
887.    // 参数示例：Math.PI / 4
888.    rotation = { x: 0, y: Math.PI / 2.3, z: 0 };
889.    label_position = { x: 0, y: 6, z: 0 };
890.    label_scale = { x: 0.1, y: 0.1, z: 0.1 };
891.    addModelAndLable('hanging_light/scene.gltf',
892.      position, scale, rotation,
893.      'threejs-tag-8',
894.      label_position, label_scale,
895.      loader,
896.      CSS3DSprite,
897.      new THREE.Group(), scene,
898.      TransformControls, camera, controls, renderer
899.    );
900.
901.    // 后期处理效果器
902.    const composer = new EffectComposer(renderer);
903.    const renderPass = new RenderPass(scene, camera);
904.    composer.addPass(renderPass);
905.
906.    const outlinePass = new OutlinePass(new THREE.Vector2(c
   ontainerWidth, containerHeight), scene, camera);
907.    outlinePass.visibleEdgeColor.set("#FFFF00");
908.    outlinePass.hiddenEdgeColor.set("gray");
909.    outlinePass.edgeGlow = 0.0;
910.    outlinePass.usePatternTexture = false;
911.    outlinePass.edgeThickness = 3.0;
912.    outlinePass.edgeStrength = 6.0;
913.    outlinePass.downSampleRatio = 2;
914.    outlinePass.pulsePeriod = 0;
915.    composer.addPass(outlinePass);
916.
917.    const outputPass = new OutputPass();
918.    composer.addPass(outputPass);
919.
920.    const smaaPass = new SMAAPass(containerWidth * renderer.
   getPixelRatio(), containerHeight * renderer.getPixelRatio())
   ;
921.    composer.addPass(smaaPass);
922.
```

```
923.    // Resize handling for container
924.    window.onresize = function () {
925.      const newWidth = container.offsetWidth;
926.      const newHeight = container.offsetHeight;
927.
928.      camera.aspect = newWidth / newHeight;
929.      camera.updateProjectionMatrix();
930.
931.      renderer.setSize(newWidth, newHeight);
932.      css3Render.setSize(newWidth, newHeight);
933.    };
934.
935.    function animate() {
936.      requestAnimationFrame(animate);
937.      controls.update();
938.      renderer.render(scene, camera);
939.      css3Render.render(scene, camera);
940.      // composer.render();
941.    }
942.    animate();
943.  });
944.  </script>
945.  <style scoped>
946.  /* Three.js 标签样式 */
947.  .threejs-label {
948.    border: 2px solid rgb(0, 255, 225);
949.    background: rgba(52, 52, 52, 0.354);
950.    padding: 10px;
951.    opacity: 0.8;
952.    z-index: 2;
953.    color: #fff;
954.  }
955.  /* Three.js 容器样式 */
956.  #threejs-container {
957.    position: absolute;
958.    top:0;
959.    width: 100vw;
960.    height: 100vh;
961.    z-index: 1;
962.  }
963.  .align-center {
964.    display: flex;
965.    align-items: center;
966.    justify-content: center;
```

```
967.    text-align: center;
968.  }
969.  .transparent {
970.    background: rgba(255, 255, 255, 0);
971.    color: #fff;
972.  }
973.  .text-very-big-font {
974.    font-size: 50px;
975.  }
976.  .text-big-font {
977.    font-size: 31px;
978.  }
979.  .text-middle-font {
980.    font-size: 27px;
981.  }
982.  .text-base-style {
983.    font-weight: bold;
984.    display: flex;
985.    align-items: center;
986.    justify-content: center;
987.    text-align: center;
988.    height: 5vh;
989.  }
990.  .text-title {
991.    letter-spacing: 0.6vh;
992.  }
993.  .blue-text {
994.    background: #00fcd6;
995.    -webkit-background-clip: text;
996.    -webkit-text-fill-color: transparent;
997.  }
998.  .gray-text {
999.    background: #cfcfcf;
1000.   -webkit-background-clip: text;
1001.   -webkit-text-fill-color: transparent;
1002.
1003. }
1004. .gradient-text {
1005.   background: linear-gradient(180deg, #d6ecff, #00b7f9);
1006.   -webkit-background-clip: text;
1007.   -webkit-text-fill-color: transparent;
1008. }
1009. .gradient-background {
1010.   /* Full height of the viewport */
```

```
  background: linear-gradient(180deg, rgba(46, 67, 90, 0.217), rgba(0, 4, 255, 0.441));
}
.transparent-background {
  background: rgba(255, 255, 255, 0);
}
.container-space-between {
  display: flex;
  justify-content: space-between;
  /* 将内容分散对齐 */
  align-items: center;
  /* 垂直居中 */
  flex-wrap: nowrap;
}
</style>
```

4.5.8 前端数据分析页面

```
<template>
  <div class="min-h-screen bg-gray-50">
    <div class="w-full mx-auto p-6">
      <h1 class="text-center text-2xl font-bold mb-6">历史数据展示与导出</h1>
      <div class="flex justify-around mb-6">
        <div class="info-box">
          <span>当前温度</span>
          <strong>9.3°C</strong>
        </div>
        <div class="info-box">
          <span>当前湿度</span>
          <strong>79%</strong>
        </div>
        <div class="info-box">
          <span>设备 ID</span>
          <strong3>ID2</strong3>
        </div>
        <div class="info-box">
          <span>在线状态</span>
          <strong2>在线</strong2>
        </div>
      </div>

      <!-- 添加图表容器 -->
```

```
25.      <div class="charts-container flex justify-between gap
   -8 mb-6">
26.        <div class="chart-wrapper w-[48%]">
27.          <Chart1
28.            :chartData="chart1Data"
29.          />
30.          <div class="mt-2">
31.            <input
32.              type="file"
33.              ref="chart1FileInput"
34.              @change="handleChart1Upload"
35.              accept=".xlsx,.xls"
36.              class="hidden"
37.            />
38.            <button class="chart-import-button" @click="$re
   fs.chart1FileInput.click()">
39.              导入温度数据
40.            </button>
41.          </div>
42.        </div>
43.        <div class="chart-wrapper w-[48%]">
44.          <Chart2
45.            :chartData="chart2Data"
46.          />
47.          <div class="mt-2">
48.            <input
49.              type="file"
50.              ref="chart2FileInput"
51.              @change="handleChart2Upload"
52.              accept=".xlsx,.xls"
53.              class="hidden"
54.            />
55.            <button class="chart-import-button" @click="$re
   fs.chart2FileInput.click()">
56.              导入湿度数据
57.            </button>
58.          </div>
59.        </div>
60.      </div>
61.
62.      <!-- 加载指示器 -->
63.      <div v-if="isLoading" class="text-center text-gray-70
   0">
64.        正在加载数据...
```

```
65.       </div>
66.
67.       <!-- 错误消息 -->
68.       <div v-if="error" class="text-center text-red-500 mb-
   4">
69.         {{ error }}
70.       </div>
71.
72.       <!-- 数据表格 -->
73.       <div v-if="!isLoading" class="bg-white rounded-lg sha
   dow p-6 w-full">
74.         <div class="table-header mb-4">
75.           <span class="text-lg font-medium">总记录
   数: {{ tableData.length }}</span>
76.         </div>
77.         <table class="w-full">
78.           <thead class="bg-gray-50">
79.             <tr>
80.               <th class="px-6 py-4 text-left text-sm font-m
   edium text-gray-500">设备 ID</th>
81.               <th class="px-6 py-4 text-left text-sm font-m
   edium text-gray-500">温度</th>
82.               <th class="px-6 py-4 text-left text-sm font-m
   edium text-gray-500">湿度</th>
83.               <th class="px-6 py-4 text-left text-sm font-m
   edium text-gray-500">时间</th>
84.             </tr>
85.           </thead>
86.           <tbody class="divide-y divide-gray-200">
87.             <tr v-for="(row, index) in paginatedData" :key=
   "index">
88.               <td class="px-6 py-4 text-sm text-gray-900">{
   { row.DeviceId }}</td>
89.               <td class="px-6 py-4 text-sm text-gray-900">{
   { row.Temperature }}</td>
90.               <td class="px-6 py-4 text-sm text-gray-900">{
   { row.Humidity }}</td>
91.               <td class="px-6 py-4 text-sm text-gray-900">{
   { row.Timestamp }}</td>
92.             </tr>
93.             <!-- 当没有数据时显示提示 -->
94.             <tr v-if="!tableData.length">
95.               <td colspan="4" class="px-6 py-4 text-center
   text-gray-500">暂无数据可显示。</td>
```

```
96.             </tr>
97.           </tbody>
98.         </table>
99.         <div class="pagination mt-4 flex justify-center">
100.           <button
101.             @click="prevPageGroup"
102.             :disabled="pageGroupStart === 1"
103.             class="nav-button"
104.             aria-label="上一组"
105.           >
106.             &lt;
107.           </button>
108.           <button
109.             v-for="page in visiblePages"
110.             :key="page"
111.             :class="{ active: currentPage === page }"
112.             @click="goToPage(page)"
113.             class="page-button"
114.           >
115.             {{ page }}
116.           </button>
117.           <button
118.             @click="nextPageGroup"
119.             :disabled="pageGroupEnd === totalPages"
120.             class="nav-button"
121.             aria-label="下一组"
122.           >
123.             &gt;
124.           </button>
125.         </div>
126.         <div class="button-group mt-4 flex justify-center
    gap-4">
127.           <button class="export-button" @click="exportRep
   ort">导出报表</button>
128.           <div class="relative">
129.             <input
130.               type="file"
131.               ref="fileInput"
132.               @change="handleFileUpload"
133.               accept=".xlsx,.xls"
134.               class="hidden"
135.             />
136.             <button class="import-button" @click="$refs.f
   ileInput.click()">
```

```
                导入 Excel
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  </template>


  <script>
  import * as XLSX from 'xlsx';
  import { saveAs } from 'file-saver';
  import Chart1 from '../components/chart1.vue'
  import Chart2 from '../components/chart2.vue'

  export default {
    name: 'App',
    components: {
      Chart1,
      Chart2
    },
    data() {
      return {
        tableData: [],
        currentPage: 1,
        itemsPerPage: 10, // 每页显示的记录数
        maxPageButtons: 5, // 最大显示的页码按钮数
        pageGroupStart: 1, // 当前页码组的起始页
        pageGroupEnd: 5, // 当前页码组的结束页
        isLoading: false, // 加载状态
        error: null, // 错误消息
        chart1Data: [], // 温度图表数据
        chart2Data: [], // 湿度图表数据
      };
    },
    computed: {
      totalPages() {
        return Math.ceil(this.tableData.length / this.items
  PerPage);
      },
      paginatedData() {
        const start = (this.currentPage - 1) * this.itemsPe
  rPage;
```

```
179.         const end = start + this.itemsPerPage;
180.         return this.tableData.slice(start, end);
181.       },
182.       visiblePages() {
183.         // 确保页码组的结束页不超过总页数
184.         if (this.pageGroupEnd > this.totalPages) {
185.           this.pageGroupEnd = this.totalPages;
186.           this.pageGroupStart = Math.max(this.pageGroupEnd
    - this.maxPageButtons + 1, 1);
187.         }
188.
189.         // 确保页码组的起始页不小于1
190.         if (this.pageGroupStart < 1) {
191.           this.pageGroupStart = 1;
192.           this.pageGroupEnd = Math.min(this.maxPageButtons,
     this.totalPages);
193.         }
194.
195.         const pages = [];
196.         for (let i = this.pageGroupStart; i <= this.pageGro
    upEnd; i++) {
197.           pages.push(i);
198.         }
199.         return pages;
200.       },
201.     },
202.     methods: {
203.       // 从 API 获取数据
204.       async fetchData() {
205.         this.isLoading = true;
206.         this.error = null;
207.         try {
208.           const response = await fetch('http://localhost:50
    00/realities/data'); // 替换为您的 API 端点
209.           if (!response.ok) {
210.             throw new Error(`网络响应不是
    OK: ${response.statusText}`);
211.           }
212.           const data = await response.json();
213.           // 假设 API 返回一个包含所需字段的对象数组
214.           this.tableData = data.map(item => ({
215.             DeviceId: item.DeviceId,
216.             Temperature: item.Temperature,
217.             Humidity: item.Humidity,
```

```
218.             Timestamp: item.Timestamp,
219.           }));
220.           this.currentPage = 1; // 重置到第一页
221.           this.updatePageGroup();
222.         } catch (error) {
223.           console.error('导入初始数据失败:', error);
224.           this.error = '无法导入初始数据，请稍后再试。';
225.         } finally {
226.           this.isLoading = false;
227.         }
228.       },
229.       // 导出当前表格数据为报表
230.       exportReport() {
231.         if (!this.tableData.length) {
232.           alert('没有数据可导出');
233.           return;
234.         }
235.
236.         // 准备要导出的数据
237.         const exportData = [
238.           ['设备 ID', '温度', '湿度', '时间'],
239.           ...this.tableData.map((row) => [
240.             row.DeviceId,
241.             row.Temperature,
242.             row.Humidity,
243.             row.Timestamp,
244.           ]),
245.         ];
246.
247.         const worksheet = XLSX.utils.aoa_to_sheet(exportDat
    a);
248.         const workbook = XLSX.utils.book_new();
249.         XLSX.utils.book_append_sheet(workbook, worksheet, '
    Report');
250.
251.         // 生成 Excel 文件并下载
252.         const wbout = XLSX.write(workbook, {
253.           bookType: 'xlsx',
254.           type: 'array',
255.         });
256.         const blob = new Blob([wbout], { type: 'application
    /octet-stream' });
257.         saveAs(blob, 'report.xlsx');
258.       },
```

```
259.      // 跳转到指定页
260.      goToPage(page) {
261.        if (page < 1) page = 1;
262.        if (page > this.totalPages) page = this.totalPages;
263.        this.currentPage = page;
264.        this.updatePageGroup();
265.      },
266.      // 跳转到上一页码组
267.      prevPageGroup() {
268.        const newStart = this.pageGroupStart - this.maxPage
    Buttons;
269.        if (newStart >= 1) {
270.          this.pageGroupStart = newStart;
271.          this.pageGroupEnd = newStart + this.maxPageButton
    s - 1;
272.          this.currentPage = this.pageGroupStart;
273.        }
274.      },
275.      // 跳转到下一页码组
276.      nextPageGroup() {
277.        const newEnd = this.pageGroupEnd + this.maxPageButt
    ons;
278.        if (newEnd <= this.totalPages) {
279.          this.pageGroupStart = newEnd - this.maxPageButton
    s + 1;
280.          this.pageGroupEnd = newEnd;
281.          this.currentPage = this.pageGroupStart;
282.        } else {
283.          this.pageGroupEnd = this.totalPages;
284.          this.pageGroupStart = Math.max(this.pageGroupEnd
    - this.maxPageButtons + 1, 1);
285.          this.currentPage = this.pageGroupStart;
286.        }
287.      },
288.      // 更新页码组
289.      updatePageGroup() {
290.        if (this.currentPage > this.pageGroupEnd) {
291.          this.pageGroupStart = this.pageGroupEnd + 1;
292.          this.pageGroupEnd = this.pageGroupStart + this.ma
    xPageButtons - 1;
293.        } else if (this.currentPage < this.pageGroupStart)
    {
294.          this.pageGroupEnd = this.pageGroupStart - 1;
```

```
295.           this.pageGroupStart = Math.max(this.pageGroupEnd
   - this.maxPageButtons + 1, 1);
296.         }
297.
298.         // 确保不超过总页数
299.         if (this.pageGroupEnd > this.totalPages) {
300.           this.pageGroupEnd = this.totalPages;
301.           this.pageGroupStart = Math.max(this.pageGroupEnd
   - this.maxPageButtons + 1, 1);
302.         }
303.
304.         // 确保起始页不小于1
305.         if (this.pageGroupStart < 1) {
306.           this.pageGroupStart = 1;
307.           this.pageGroupEnd = Math.min(this.maxPageButtons,
    this.totalPages);
308.         }
309.       },
310.       async handleFileUpload(event) {
311.         const file = event.target.files[0];
312.         if (!file) return;
313.
314.         try {
315.           const reader = new FileReader();
316.           reader.onload = (e) => {
317.             const data = new Uint8Array(e.target.result);
318.             const workbook = XLSX.read(data, { type: 'array
   ' });
319.
320.             // 假设第一个工作表包含数据
321.             const worksheet = workbook.Sheets[workbook.Shee
   tNames[0]];
322.             const jsonData = XLSX.utils.sheet_to_json(works
   heet);
323.
324.             // 更新表格数据
325.             this.tableData = jsonData.map(row => ({
326.               DDeviceId: row['设备 ID'],
327.               Temperature: row['温度'],
328.               Humidity: row['湿度'],
329.               Timestamp: row['时间']
330.             }));
331.
332.             // 更新图表数据
```

```
333.             this.temperatureChartData = jsonData.map(row =>
    [
334.               row['时间'],
335.               parseFloat(row['温度'])
336.             ]);
337.
338.             this.humidityChartData = jsonData.map(row => [
339.               row['时间'],
340.               parseFloat(row['湿度'])
341.             ]);
342.
343.             // 重置分页
344.             this.currentPage = 1;
345.             this.updatePageGroup();
346.           };
347.           reader.readAsArrayBuffer(file);
348.         } catch (error) {
349.           console.error('文件读取错误:', error);
350.           this.error = '文件读取失败，请确保文件格式正确';
351.         }
352.       },
353.       // 处理温度图表数据上传
354.       async handleChart1Upload(event) {
355.         const file = event.target.files[0];
356.         if (!file) return;
357.
358.         try {
359.           const reader = new FileReader();
360.           reader.onload = (e) => {
361.             const data = new Uint8Array(e.target.result);
362.             const workbook = XLSX.read(data, { type: 'array
    ' });
363.             const worksheet = workbook.Sheets[workbook.Shee
    tNames[0]];
364.             const jsonData = XLSX.utils.sheet_to_json(works
    heet);
365.
366.             // 更新图表数据
367.             this.chart1Data = jsonData.map(row => [
368.               row['时间'] || row['Time'] || '',
369.               parseFloat(row['温度
    '] || row['Temperature'] || 0)
370.             ]).filter(item => !isNaN(item[1])); // 过滤掉无
    效数据
```

```
371.           };
372.           reader.readAsArrayBuffer(file);
373.         } catch (error) {
374.           console.error('温度数据文件读取错误:', error);
375.           this.error = '温度数据文件读取失败，请确保文件格式正
   确';
376.         }
377.       },
378.
379.       // 处理湿度图表数据上传
380.       async handleChart2Upload(event) {
381.         const file = event.target.files[0];
382.         if (!file) return;
383.
384.         try {
385.           const reader = new FileReader();
386.           reader.onload = (e) => {
387.             const data = new Uint8Array(e.target.result);
388.             const workbook = XLSX.read(data, { type: 'array
   ' });
389.             const worksheet = workbook.Sheets[workbook.Shee
   tNames[0]];
390.             const jsonData = XLSX.utils.sheet_to_json(works
   heet);
391.
392.             // 更新图表数据
393.             this.chart2Data = jsonData.map(row => [
394.               row['时间'] || row['Time'] || '',
395.               parseFloat(row['湿度
   '] || row['Humidity'] || 0)
396.             ]).filter(item => !isNaN(item[1])); // 过滤掉无
   效数据
397.           };
398.           reader.readAsArrayBuffer(file);
399.         } catch (error) {
400.           console.error('湿度数据文件读取错误:', error);
401.           this.error = '湿度数据文件读取失败，请确保文件格式正
   确';
402.         }
403.       },
404.     },
405.     mounted() {
406.       // 页面加载时自动从 API 获取数据
407.       this.fetchData();
```

```
  },
};
</script>

<style scoped>
body, html {
  margin: 0;
  padding: 0;
  height: 100%;
  width: 100%;
}

.min-h-screen {
  min-height: 100vh;
  min-width: 100vw;
}

.w-full {
  width: 100%;
}

.mx-auto {
  margin-left: auto;
  margin-right: auto;
}

.p-6 {
  padding: 1.5rem;
}

.mb-6 {
  margin-bottom: 1.5rem;
}

.bg-gray-50 {
  background-color: #f9fafb;
}

.bg-white {
  background-color: #ffffff;
}

.rounded-lg {
  border-radius: 0.5rem;
```

```
452.  }
453.
454.  .shadow {
455.    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
456.  }
457.
458.  .text-center {
459.    text-align: center;
460.  }
461.
462.  .text-2xl {
463.    font-size: 1.5rem;
464.  }
465.
466.  .font-bold {
467.    font-weight: 700;
468.  }
469.
470.  .text-lg {
471.    font-size: 1.125rem;
472.  }
473.
474.  .font-medium {
475.    font-weight: 500;
476.  }
477.
478.  .block {
479.    display: block;
480.  }
481.
482.  .divide-y > :not([hidden]) ~ :not([hidden]) {
483.    border-top-width: 1px;
484.  }
485.
486.  .divide-gray-200 {
487.    --tw-divide-opacity: 1;
488.    border-color: rgba(229, 231, 235, var(--tw-divide-opaci
    ty));
489.  }
490.
491.  .text-sm {
492.    font-size: 0.875rem;
493.  }
494.
```

```
.text-gray-500 {
  color: #6b7280;
}

.text-gray-900 {
  color: #111827;
}

.mt-4 {
  margin-top: 1rem;
}

.flex {
  display: flex;
}

.justify-center {
  justify-content: center;
}

.nav-button {
  margin: 0 5px;
  padding: 8px 16px;
  border: none;
  background-color: #409eff;
  color: white;
  cursor: pointer;
  border-radius: 4px;
  font-size: 18px;
  transition: background-color 0.3s;
}

.nav-button:hover:not(:disabled) {
  background-color: #66b1ff;
}

.nav-button:disabled {
  background-color: #cccccc;
  cursor: not-allowed;
}

.page-button {
  margin: 0 5px;
  padding: 8px 12px;
```

```
    border: none;
    background-color: #f0f0f0;
    color: #333333;
    cursor: pointer;
    border-radius: 4px;
    font-size: 16px;
    transition: background-color 0.3s, color 0.3s;
}
.info-box {
    background-color: #ffffff;
    border-radius: 0.3rem;
    padding: 2rem;
    /* text-align: center; */
    width: 20%;
    margin: 0 3rem;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

.info-box span {
    display: block;
    font-size: 0.875rem;
    color: #6b7280;
}

.info-box strong {
    display: block;
    font-size: 1.5rem;
    margin-top: 0.5rem;
    font-weight: 800;
}
.page-button:hover {
    background-color: #ddd;
}
.info-box strong2 {
    display: block;
    font-size: 1.5rem;
    margin-top: 0.5rem;
    font-weight: 800;
    color: #45c769;
}
.info-box strong3 {
    display: block;
    font-size: 1.5rem;
    margin-top: 0.5rem;
```

```
    font-weight: 800;
    color: #FF0000;
  }
  .page-button:hover {
    background-color: #ddd;
  }

  .page-button.active {
    background-color: #1a8cff;
    color: white;
  }

  .export-button {
    display: block;
    margin: 0 auto;
    padding: 14px 28px;
    background-color: #28a745;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 6px;
    font-size: 18px;
    transition: background-color 0.3s;
  }

  .export-button:hover {
    background-color: #45c769;
  }

  .button-group {
    display: flex;
    gap: 1rem;
    justify-content: center;
  }

  .import-button {
    padding: 14px 28px;
    background-color: #409EFF;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 6px;
    font-size: 18px;
    transition: background-color 0.3s;
```

```
627.  }
628.
629.  .import-button:hover {
630.    background-color: #66b1ff;
631.  }
632.
633.  .hidden {
634.    display: none;
635.
636.
637.  }
638.
639.  .charts-container {
640.    background-color: white;
641.    padding: 20px;
642.    border-radius: 8px;
643.    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
644.    margin-bottom: 20px;
645.    margin-left: 50px;
646.    margin-right: 100px;
647.    gap: 30px;
648.  }
649.
650.  .chart-wrapper {
651.    background-color: #fff;
652.    padding: 15px;
653.    border-radius: 6px;
654.    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
655.    margin: 0 20px;
656.  }
657.
658.  .chart-import-button {
659.    padding: 8px 16px;
660.    background-color: #409EFF;
661.    color: white;
662.    border: none;
663.    cursor: pointer;
664.    border-radius: 4px;
665.    font-size: 14px;
666.    transition: background-color 0.3s;
667.    margin-top: 10px;
668.  }
669.
670.  .chart-import-button:hover {
```

```
    background-color: #66b1ff;
  }
  </style>
```

4.5.9 温度折线图

```
<!-- src/components/ForecastChart.vue -->
<template>
  <div>
    <h2 class="text-center text-xl font-semibold mb-4">历史趋势图—温度</h2>
    <canvas ref="forecastChart"></canvas>
    <div v-if="isLoading" class="text-center mt-4 text-gray-500">
      数据加载中...
    </div>
    <div v-if="error" class="text-center mt-4 text-red-500">
      {{ error }}
    </div>
  </div>
</template>

<script>
import { ref, onMounted } from "vue";
import {
  Chart,
  LineController,
  LineElement,
  PointElement,
  LinearScale,
  Title,
  CategoryScale,
  Tooltip,
  Legend,
} from "chart.js";

Chart.register(
  LineController,
  LineElement,
  PointElement,
  LinearScale,
```

```
34.   CategoryScale,
35.   Title,
36.   Tooltip,
37.   Legend
38. );
39. 
40. export default {
41.   name: "ForecastChart",
42.   setup() {
43.     const forecastChart = ref(null);
44.     let chartInstance = null;
45. 
46.     // 数据状态
47.     const isLoading = ref(false);
48.     const error = ref(null);
49.     const chartData = ref([]);
50. 
51.     // 从后端获取数据
52.     const fetchChartData = async () => {
53.       isLoading.value = true;
54.       error.value = null;
55.       try {
56.         const response = await fetch("http://localhost:5000
    /realities/temperature"); // 替换为你的后端 API 路径
57.         if (!response.ok) {
58.           throw new Error(`网络响应错
    误: ${response.statusText}`);
59.         }
60.         const data = await response.json();
61.         chartData.value = data.map((item) => [item.time, it
    em.temperature]); // 处理返回数据
62.       } catch (err) {
63.         error.value = `数据加载失败: ${err.message}`;
64.       } finally {
65.         isLoading.value = false;
66.       }
67.     };
68. 
69.     // 绘制图表
70.     const renderChart = () => {
71.       const labels = chartData.value.map((item) => item[0]);
     // 第一列作为X轴（时间）
72.       const dataPoints = chartData.value.map((item) => item
    [1]); // 第二列作为Y轴（温度）
```

```
73.
74.      if (chartInstance) {
75.        chartInstance.destroy(); // 销毁旧图表实例
76.      }
77.
78.      chartInstance = new Chart(forecastChart.value, {
79.        type: "line",
80.        data: {
81.          labels: labels,
82.          datasets: [
83.            {
84.              label: "温度数据",
85.              data: dataPoints,
86.              borderColor: "rgba(255, 99, 132, 1)",
87.              backgroundColor: "rgba(255, 99, 132, 0.2)",
88.              fill: true,
89.              tension: 0.4,
90.            },
91.          ],
92.        },
93.        options: {
94.          responsive: true,
95.          plugins: {
96.            legend: {
97.              position: "top",
98.            },
99.            title: {
100.                display: false,
101.                text: "历史趋势图—温度",
102.              },
103.            },
104.          },
105.        });
106.      };
107.
108.      // 初始化逻辑
109.      onMounted(async () => {
110.        await fetchChartData(); // 获取数据
111.        renderChart(); // 绘制图表
112.      });
113.
114.      return {
115.        forecastChart,
116.        isLoading,
```

```
        error,
      };
    },
  };
  </script>

  <style scoped>
  canvas {
    max-width: 100%;
    height: 400px;
  }
  </style>

```

4.5.10 湿度折线图

```
<!-- src/components/HistoricalChart.vue -->
<template>
  <div>
    <h2 class="text-center text-xl font-semibold mb-4">历史趋势图—湿度</h2>
    <canvas ref="historicalChart"></canvas>
    <div v-if="isLoading" class="text-center mt-4 text-gray-500">
      数据加载中...
    </div>
    <div v-if="error" class="text-center mt-4 text-red-500">
      {{ error }}
    </div>
  </div>
</template>

<script>
import { ref, onMounted } from "vue";
import {
  Chart,
  LineController,
  LineElement,
  PointElement,
  LinearScale,
  Title,
```

```
  CategoryScale,
  Tooltip,
  Legend,
} from "chart.js";

Chart.register(
  LineController,
  LineElement,
  PointElement,
  LinearScale,
  CategoryScale,
  Title,
  Tooltip,
  Legend
);

export default {
  name: "HistoricalChart",
  setup() {
    const historicalChart = ref(null);
    let chartInstance = null;

    // 数据状态
    const isLoading = ref(false);
    const error = ref(null);
    const chartData = ref([]);

    // 从后端获取湿度数据
    const fetchChartData = async () => {
      isLoading.value = true;
      error.value = null;
      try {
        const response = await fetch("http://localhost:5000/realities/humidity"); // 替换为你的后端 API 路径
        if (!response.ok) {
          throw new Error(`网络响应错误: ${response.statusText}`);
        }
        const data = await response.json();
        chartData.value = data.map((item) => [item.time, item.humidity]); // 处理返回数据
      } catch (err) {
        error.value = `数据加载失败: ${err.message}`;
      } finally {
```

```
65.         isLoading.value = false;
66.       }
67.     };
68.
69.     // 绘制图表
70.     const renderChart = () => {
71.       const labels = chartData.value.map((item) => item[0]);
    // 第一列作为X轴（时间）
72.       const dataPoints = chartData.value.map((item) => item
   [1]); // 第二列作为Y轴（湿度）
73.
74.       if (chartInstance) {
75.         chartInstance.destroy(); // 销毁旧图表实例
76.       }
77.
78.       chartInstance = new Chart(historicalChart.value, {
79.         type: "line",
80.         data: {
81.           labels: labels,
82.           datasets: [
83.             {
84.               label: "湿度数据",
85.               data: dataPoints,
86.               borderColor: "rgba(75, 192, 192, 1)",
87.               backgroundColor: "rgba(75, 192, 192, 0.2)",
88.               fill: true,
89.               tension: 0.4,
90.             },
91.           ],
92.         },
93.         options: {
94.           responsive: true,
95.           plugins: {
96.             legend: {
97.               position: "top",
98.             },
99.             title: {
100.                display: false,
101.                text: "历史趋势图—湿度",
102.              },
103.            },
104.          },
105.        });
106.      };
```

```
107.
108.     // 初始化逻辑
109.     onMounted(async () => {
110.       await fetchChartData(); // 获取数据
111.       renderChart(); // 绘制图表
112.     });
113.
114.     return {
115.       historicalChart,
116.       isLoading,
117.       error,
118.     };
119.   },
120. };
121. </script>
122.
123. <style scoped>
124. canvas {
125.   max-width: 100%;
126.   height: 400px;
127. }
128. </style>
```

4.5.9 前端密码修改页面

```
1. <template>
2.     <v-container>
3.       <v-sheet class="pa-12" rounded color="#F7F7F7">
4.         <v-card class="mx-auto px-6 py-9" max-width="344" e
   levation="4">
5.           <template v-slot:title>
6.             修改密码
7.           </template>
8.
9.           <v-form v-model="form" @submit.prevent="onSubmit"
   >
10.            <!-- 当前密码输入框 -->
11.            <v-text-field
12.              v-model="currentPassword"
13.              :readonly="loading"
14.              :rules="currentPasswordRules"
15.              class="mb-2"
```

```
16.               label="当前密码"
17.               clearable
18.               type="password"
19.               outlined
20.               dense
21.             >
22.               <template v-slot:prepend>
23.                 <v-icon icon="mdi-lock"></v-icon>
24.               </template>
25.             </v-text-field>
26.
27.             <!-- 新密码输入框 -->
28.             <v-text-field
29.               v-model="newPassword"
30.               :readonly="loading"
31.               :rules="newPasswordRules"
32.               class="mb-2"
33.               label="新密码"
34.               clearable
35.               type="password"
36.               outlined
37.               dense
38.             >
39.               <template v-slot:prepend>
40.                 <v-icon icon="mdi-lock-reset"></v-icon>
41.               </template>
42.             </v-text-field>
43.
44.             <!-- 确认新密码输入框 -->
45.             <v-text-field
46.               v-model="confirmPassword"
47.               :readonly="loading"
48.               :rules="confirmPasswordRules"
49.               class="mb-2"
50.               label="确认新密码"
51.               clearable
52.               type="password"
53.               outlined
54.               dense
55.             >
56.               <template v-slot:prepend>
57.                 <v-icon icon="mdi-lock-check"></v-icon>
58.               </template>
59.             </v-text-field>
```

```
60.
61.            <v-btn :loading="loading" color="success" size=
   "large" type="submit" variant="elevated" block>
62.              提交
63.            </v-btn>
64.          </v-form>
65.        </v-card>
66.      </v-sheet>
67.    </v-container>
68.  </template>
69.
70.  <script setup>
71.  import { useRouter } from 'vue-router';
72.  import { ref } from 'vue';
73.
74.  const router = useRouter();
75.
76.  // 密码输入
77.  const currentPassword = ref('');
78.  const newPassword = ref('');
79.  const confirmPassword = ref('');
80.
81.  // 表单验证规则
82.  const currentPasswordRules = ref([
83.    value => value?.length > 0 || '当前密码未输入',
84.  ]);
85.
86.  const newPasswordRules = ref([
87.    value => value?.length > 0 || '新密码未输入',
88.  ]);
89.
90.  const confirmPasswordRules = ref([
91.    value => value?.length > 0 || '确认新密码未输入',
92.  ]);
93.
94.  const form = ref(false);
95.  const loading = ref(false);
96.
97.  // 提交事件
98.  function onSubmit() {
99.    if (!form.value) return;
100.
101.      loading.value = true;
102.
```

```
    // 这里可以添加实际的后端请求逻辑
    // 例如：调用 API 进行密码修改

    loading.value = false;
    router.push('/'); // 修改成功后跳转首页或其他页面
  }
  </script>

  <style scoped>
  /* 可以根据需求添加更多的样式 */
  </style>
```

五、实验运行结果展示

芯片 MQTT 接受数据：

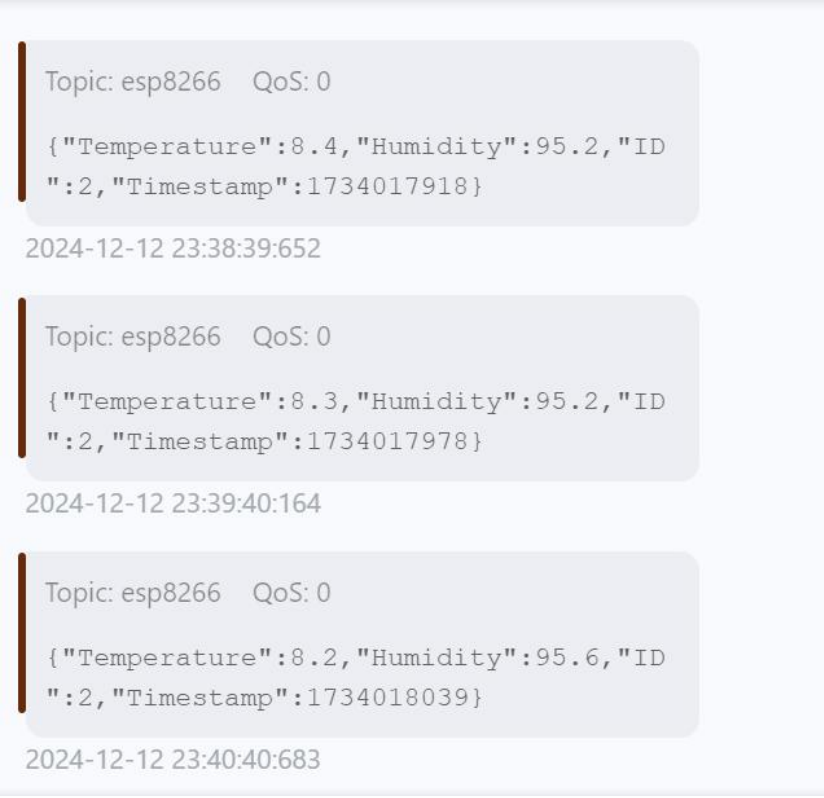


图 12 MQTT 接受数据

温湿度预测：

```
预测时间: 12:14:54, 预测温度: 16.789907455444336, 预测湿度: 75.91666412353516
预测时间: 12:19:54, 预测温度: 16.751659393310547, 预测湿度: 75.20782470703125
预测时间: 12:24:54, 预测温度: 16.717391967773438, 预测湿度: 74.63725280761719
预测时间: 12:29:54, 预测温度: 16.688446044921875, 预测湿度: 74.16345977783203
预测时间: 12:34:54, 预测温度: 16.663421630859375, 预测湿度: 73.77703094482422
预测时间: 12:39:54, 预测温度: 16.641624450683594, 预测湿度: 73.46617889404297
预测时间: 12:44:54, 预测温度: 16.622711181640625, 预测湿度: 73.2186508178711
预测时间: 12:49:54, 预测温度: 16.606443405151367, 预测湿度: 73.02275848388672
预测时间: 12:54:54, 预测温度: 16.59258460998535, 预测湿度: 72.86821746826172
预测时间: 12:59:54, 预测温度: 16.580894470214844, 预测湿度: 72.74637603759766
预测时间: 13:04:54, 预测温度: 16.571115493774414, 预测湿度: 72.65020751953125
预测时间: 13:09:54, 预测温度: 16.562992095947266, 预测湿度: 72.57415008544922
预测时间: 13:14:54, 预测温度: 16.556285858154297, 预测湿度: 72.51383972167969
预测时间: 13:19:54, 预测温度: 16.5507755279541, 预测湿度: 72.46588897705078
预测时间: 13:24:54, 预测温度: 16.546266555786133, 预测湿度: 72.42767333984375
预测时间: 13:29:54, 预测温度: 16.542587280273438, 预测湿度: 72.39714813232422
预测时间: 13:34:54, 预测温度: 16.539594650268555, 预测湿度: 72.3727035522461
预测时间: 13:39:54, 预测温度: 16.537160873413086, 预测湿度: 72.35310363769531
```

图 13 预测温湿度示意图

温湿度预测折现图：

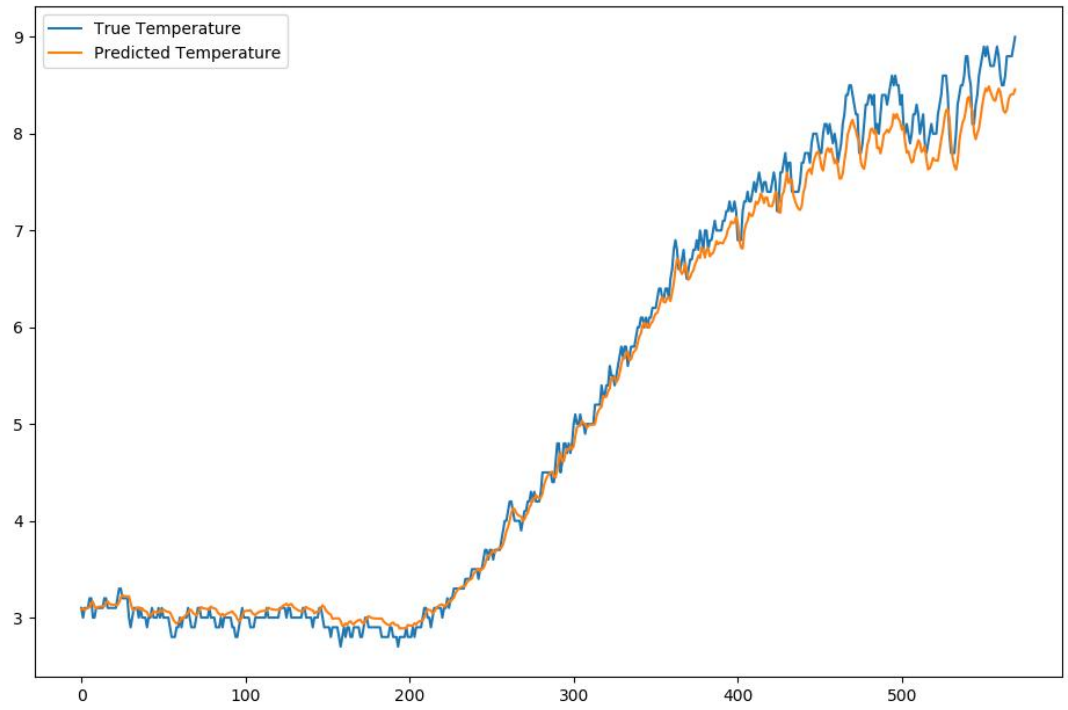


图 14 模型准确度示意图

后端运行状态图：



图 15 后端运行状态

3D 农业平台主页：

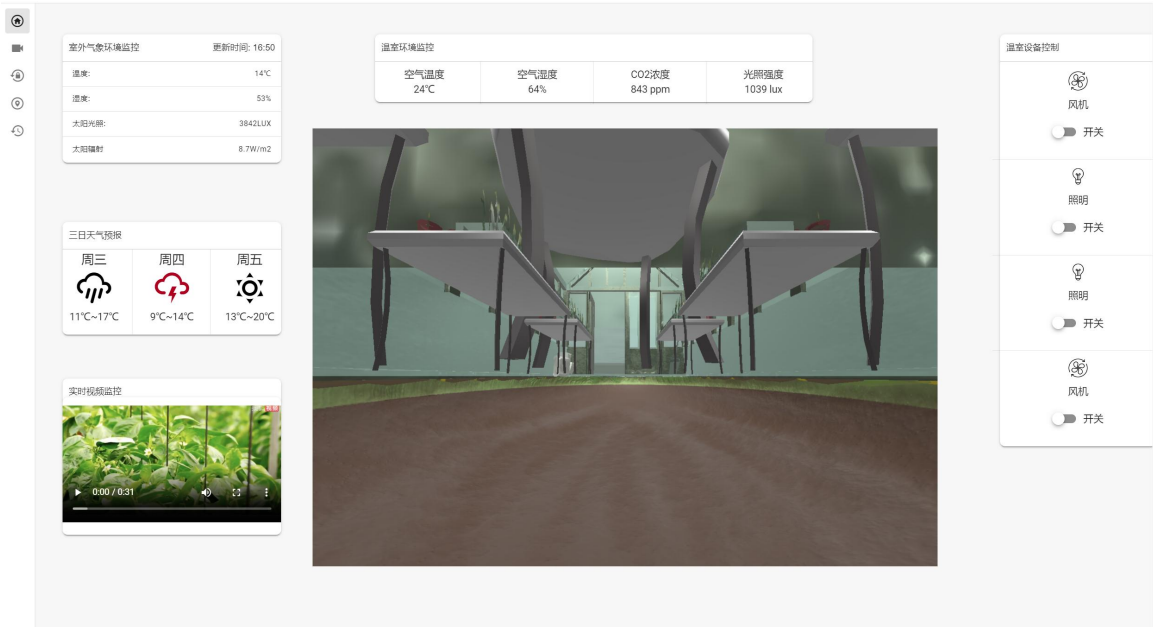


图 16 平台主页

实时监控接口：

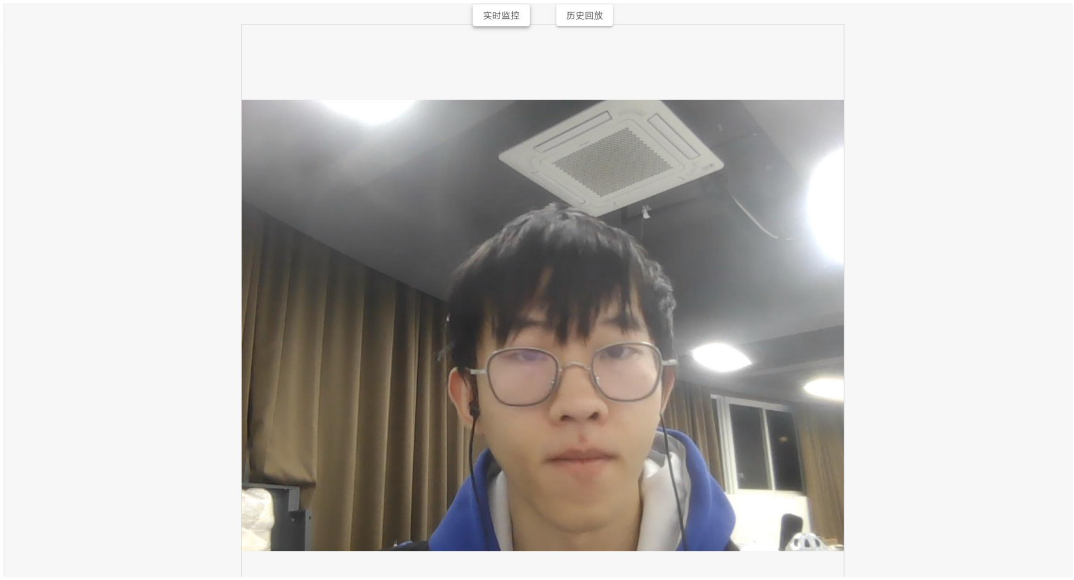


图 17 实时监控接口

3D 仿真大棚展示：

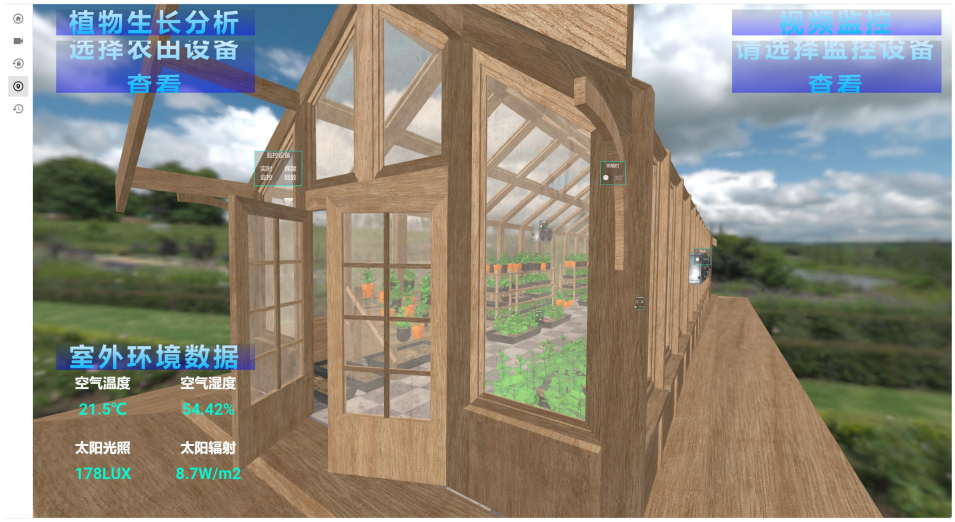


图 18 3D 模型展示



图 19 3D 模型展示

数据分析页面：



图 20 数据分析折线图

报表导出功能：

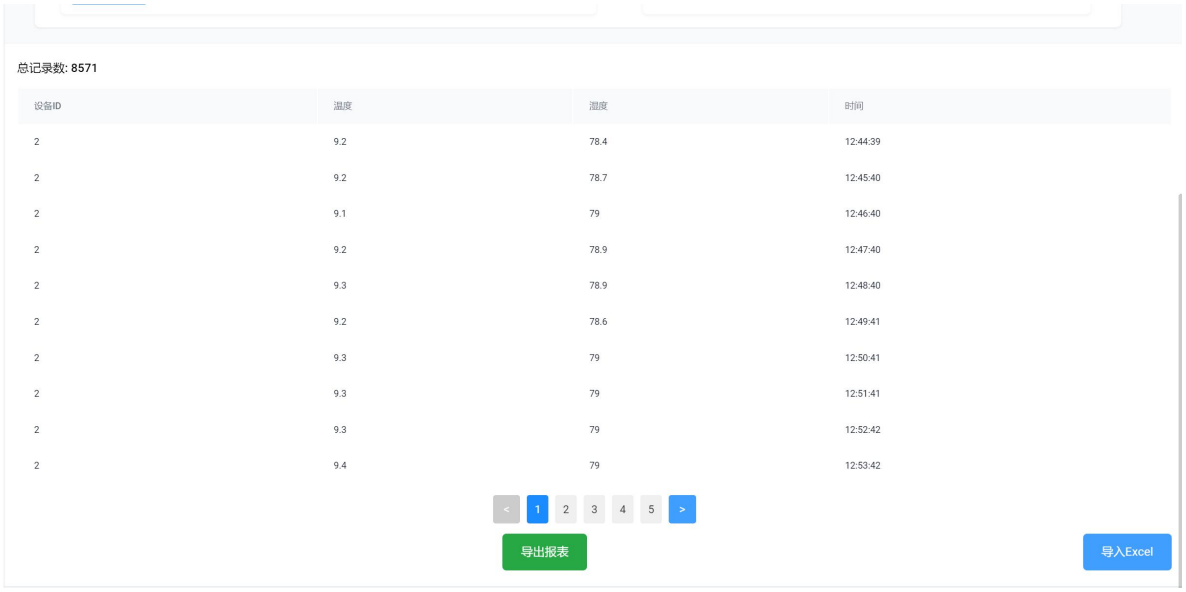


图 21 报表导出

六、实验总结

1) 系统架构和功能实现： 本项目实现了一个完整的温湿度数据采集、分析、预测与控制系统，成功地将传感器数据采集、LSTM 模型分析、设备控制、

3D 仿真大棚可视化与前端展示集成在一个系统中。通过不同的模块协同工作，系统能够实时获取数据、进行分析、预测温湿度变化并对大棚设备进行精准控制。

2) 数据采集与预处理： 系统通过传感器持续实时地采集环境数据，并对采集到的数据进行必要的预处理。数据清洗和异常值检测的步骤提高了数据质量，为后续的分析与预测提供了高质量的输入数据。即使在不稳定或异常环境下，数据预处理模块能够有效处理噪声数据，保证系统的稳定性和可靠性。

3) LSTM 模型的预测效果： LSTM 模型成功应用于温湿度数据的预测。模型训练过程涉及数据集的划分与验证，结果表明模型的预测准确性较高，能够有效预测未来的温湿度趋势。对于需要进一步优化预测准确性的场景，模型调整和参数优化能够显著提升预测结果。

4) 设备控制与反馈机制： 系统根据 LSTM 模型的预测结果生成设备控制指令，并通过实时反馈机制对大棚内部的温湿度进行调节。这一闭环控制系统表现良好，确保了大棚内设备的动态调整和高效运行，避免了不必要的能耗。

5) 3D 仿真大棚的可视化： 3D 仿真大棚功能在前端展示中成功实现，能够根据实时的温湿度数据和设备控制指令动态更新大棚内部的状态，帮助用户更直观地查看环境变化。仿真大棚不仅提供了设备状态的反馈，也提高了系统的交互性和用户体验。

6) 前端数据展示与用户交互： 系统前端通过图形化界面展示温湿度数据和设备状态，用户可以方便地查看环境变化趋势和设备运行情况。系统实时更新的功能确保了用户能够获取到最新的数据信息，增强了用户的控制感和系统的互动性。

7) 系统的可扩展性和稳定性： 系统架构采用了模块化设计，各个功能模块之间独立运行，彼此之间的耦合度较低，具有较好的可扩展性。无论是传感器的增加、LSTM 模型的优化，还是仿真大棚的功能扩展，都能通过简化的接口进行无缝集成。此外，系统也具有较高的稳定性，能够在长时间运行和高并发场景下保持正常。